# Answering Federated Range Queries with Local Differential Privacy

Yuemin Zhang†, Qingqing Ye†, Junxu Liu†, Wei Dong‡
†The Hong Kong Polytechnic University, ‡Nanyang Technological University
ymin.zhang@connect.polyu.hk, qqing.ye@polyu.edu.hk, junxu.liu@polyu.edu.hk, wei_dong@ntu.edu.sg

***Abstract*—Answering range queries under local differential privacy (LDP) is a well-studied problem, which enables service providers to gain insights into decentralized data distributions. However, existing approaches operate exclusively in a single-party setting and prove challenging to extend to practical cross-party environments. In this paper we study the problem of *federated range queries* under LDP, which addresses cross-party range queries in decentralized settings. Applying LDP to federated range queries amplifies statistical heterogeneity challenges, as utility deterioration stems from not only the accumulated LDP noise, but also the unrealistic uniform- or linear-distribution assumption in existing methods. Our solution, the private cubic spline (PrivCS) mechanism, employs a hierarchical interval tree structure to model the underlying data distribution and accurately answer arbitrary range queries, effectively addressing the problem while satisfying $\epsilon$-LDP. Additionally, we introduce the collaborative frequency filtering aggregation that adaptively calibrates and masks frequency estimates by leveraging relationships among across-party frequencies and analyzing pruning bias, thereby reducing the cumulative noise. To the best of our knowledge, this is the first study to explore federated range queries in cross-party scenarios, under the stringent LDP guarantee. Comprehensive experiments on both real-world and synthetic datasets demonstrate the effectiveness of our mechanism.**

## I. Introduction

As a fundamental and prevalent operation for computing essential statistics of user populations, the range query calculates the proportion of users whose input falls within a specified interval. Nevertheless, since individual data often contains sensitive personal information, users are typically unwilling to share their raw data.

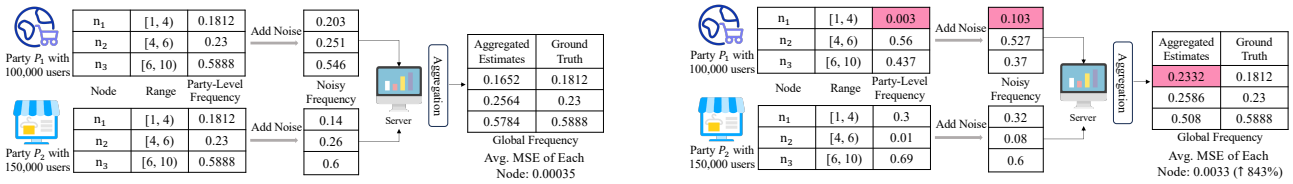
Local differential privacy (LDP) [1], the *de facto* privacy standard for decentralized settings, enables users to perturb their data locally before transmitting sanitized versions to data collectors. This makes LDP particularly well-suited for privacy-preserving data analytics. Numerous studies have explored LDP-based range queries [2], [3], [4], [5], [6], [7], [8]. However, these approaches operate exclusively in a single-party setting, where all users belong to the same party and the data collector receives sanitized data directly from them.

Practical deployments, conversely, typically involve **users distributed across multiple parties** [9], [10]. For instance, an e-commerce company requires regional branches (e.g., Europe and Asia) to collaboratively estimate the proportion of customers, who are aged 18–25 and favor sports equipment priced between \$20–\$50 during Black Friday. Such cross-party analysis is essential for strategic planning (e.g., optimizing marketing campaigns), yet remains unaddressed by existing single-party LDP solutions. This gap stems from two fundamental limitations of multi-party settings — prohibitive communication overhead and cross-party statistical heterogeneity.

Intuitively, we can let all users upload their perturbed data using an LDP protocol for frequency estimation (e.g., OUE [11]) to the central server for estimating a global noisy histogram and answering range queries. However, such a method directly leads to **prohibitive communication overhead** on the server side. Specifically, LDP mechanisms often involve encoding a single value into a vector (e.g., with a domain length $|\mathcal{X}| = 2^{20}$), resulting in high communication costs (e.g., up to $O(|\mathcal{U}| \cdot |\mathcal{X}|)$) in the server side where $|\mathcal{U}|$ is the user population size. Although more communication-efficient protocols, such as OLH [11] or $k$-RR [12], [13], can be used, their communication overheads (i.e., $O(|\mathcal{U}|)$) remain substantial. Additionally, decoding the randomized data from these mechanisms at the server requires an exhaustive scan of the entire domain $\mathcal{X}$ for each user, which is computationally infeasible for large domains [14], [15], [16].

Fortunately, a cross-party range query can be decomposed into multiple sub-tasks, each assigned to a different party. The final result is then obtained by aggregating the partial results contributed by all participants. A straightforward implementation is to apply an existing LDP-based range query algorithm (e.g., PrivNUD [2]) within each party, followed by transmitting the perturbed party-level frequencies to the central server for aggregation. However, this approach is significantly affected by **cross-party statistical heterogeneity**, which stems from the non-IID nature of decentralized data. On the one hand, such heterogeneity can lead to skewed (and noisy) internal distributions within different tree nodes or grid cells, making the uniform- or linear-distributed assumptions in prior works [2], [3], [5] prone to large approximation errors when queries intersect subdomains. On the other hand, it can undermine the reliability of party-level frequency estimates in global aggregation. For instance, when a party contributes only a marginal proportion of data within certain intervals, its local frequency estimates may be easily suppressed by LDP noise. As illustrated in Figure 1, under a non-IID setting, party $P_1$ exhibits a much lower frequency of $0.003$ in the interval $[1, 4)$, compared to $0.1812$ under IID conditions. Such a small value is easily overwhelmed by the LDP noise, resulting in a substantial increase in average error (e.g., increased by $843\%$).

In this work, we introduce the problem of *federated range queries* under LDP, which addresses cross-party range queries




(a) IID (b) Non-IID

Fig. 1: An example to illustrate data heterogeneity issue, which leads to large aggregation errors under LDP noise.

in decentralized settings. Our solution, the private cubic spline (PrivCS) mechanism, employs a hierarchical interval tree structure to capture the underlying data distribution and accurately answer arbitrary range queries. In contrast to prior work [2], [3], [5] that relies on uniform- or linear-distribution assumptions, PrivCS leverages cubic splines (CS) for distribution fitting. Each spline is parameter-efficient, requiring only a few parameters to represent the distribution within a tree node.

We adopt cubic splines (CS) thanks to their dual advantages. First, they inherently avoid the Runge phenomenon [17] compared to high-degree interpolation while delivering superior approximation accuracy over low-degree alternatives. PrivCS optimizes CS parameters to simultaneously capture the distribution's geometric properties and minimize expected approximation error. Additionally, data heterogeneity in multi-party settings amplifies instability in party-level estimates (e.g., extreme values) under LDP noise. Naive aggregation of these party-level frequencies may introduce excessive noise. To address this, PrivCS integrates a collaborative frequency filtering (CF) strategy. CF leverages cross-party frequency information to adaptively calibrate estimates using inter-party consistency, selectively prunes noise-dominated frequencies, and minimizes pruning-induced bias during global aggregation.

Our key contributions include:

- This is the first to explore stringent $\epsilon$-LDP solution to answer federated range queries over arbitrary data distribution in a cross-party setting. A novel private cubic spline mechanism is designed to model the underlying data distribution, which eliminates the dependence of assumptions on uniform or linear data distribution.
- We propose a collaborative frequency filtering aggregation that collaboratively post-processes the estimated frequencies by adaptively masking and calibrating certain frequencies to reduce the accumulated errors.
- We conduct comprehensive experiments on real-world and synthetic datasets to confirm the superiority of our mechanism in comparison with existing solutions.

The remainder of this paper is organized as follows. We discuss related work in Section II, introduce preliminaries and problem statement in Sections III and IV, and detail our solution in Section V. Then we present experimental results in Section VI, and conclude the paper in Section VII.

## II. Related Work

Differential privacy (DP) [18] and local differential privacy (LDP) [1], [19] have been widely studied for different data analysis problems. Representative tasks include SQL/SPJA query answering and multi-way join counting [20], [21], [22], graph and set-valued data analysis [23], [24], key-value data collection [25], trajectory data analysis [26], [27], and statistical estimation under LDP variants [28], [29].

Privately answering range queries is a fundamental task that has been widely studied under both DP and LDP. Under DP, prior work mainly focuses on hierarchical representations and post-processing: Hay *et al*. [30] propose the first DP hierarchical tree with constrained inference, later improved by Qardaji *et al*. [31]. Subsequent studies enhance the representation or budget allocation, including Haar wavelets [32], [33], non-uniform partitioning [34], optimized node decomposition [35], and balanced box-decomposition trees [36]. Under LDP, existing methods are typically tree-based or grid-based. Tree-based approaches build hierarchical structures with carefully chosen fan-outs [6], [4], and further improve utility via interval merging [5], adaptive fan-out [2], wavelet-based optimization [6], or piecewise-linear approximation [3]. Grid-based approaches include hybrid-dimensional grids [7] and prefix-sum cubes [8].

Existing studies under LDP primarily consider the traditional single-party setting, neglecting more practical scenarios where users' data are distributed across multiple parties. Furthermore, existing studies rely on simplistic assumptions of uniform or linear internal distributions of subdomains, which may limit their approximation capabilities and consequently result in substantial internal range approximation errors. In contrast, our PrivCS assumes the internal distribution of a node can be approximated by a cubic polynomial, which is much more realistic and milder. To the best of our knowledge, PrivCS is the first $\epsilon$-LDP framework specifically developed for answering federated range queries in cross-party settings while addressing the challenges posed by non-IID data distributions.

## III. Preliminaries

### A. Local Differential Privacy

Local Differential Privacy (LDP) [1] enables each user to perturb her data locally, without the need for a trusted data curator. It achieves plausible deniability, which is controlled by a privacy budget $\epsilon$. The following provides a formal definition of LDP, which enjoys the property of post-processing [37].

TABLE I: Notations.

| Symbols | Description |
|---|---|
| $P_i$ | The $i$-th party |
| $\|U_i\|$ | The user population of $P_i$ |
| $A^\tau$ | The $\tau$-th attribute |
| $\|\mathcal{X}^\tau\|$ | The domain size of $A^\tau$ |
| $F[j]$ | The frequency of $j$-th bin in histogram $F$ |
| $f^i$ | A node's party-level frequency estimated in $P_i$ |

**Definition 1** ($\epsilon$-LDP)**.** *Given a privacy budget $\epsilon > 0$, a randomized mechanism $\mathcal{M} : \mathcal{X} \to \mathcal{Y}$ provides $\epsilon$-LDP if and only if, for any two inputs $x, x' \in \mathcal{X}$ and any set of possible outputs $\mathcal{Y}$, the following inequality holds:*

$$\Pr[\mathcal{M}(x) \in \mathcal{Y}] \leq e^\epsilon \times \Pr[\mathcal{M}(x') \in \mathcal{Y}].$$

**Theorem 1.** *(Post-Processing) Let $\mathcal{M} : \mathcal{X} \to \mathcal{Y}$ be a mechanism that satisfies $\epsilon$-LDP, and $\mathcal{F} : \mathcal{Y} \to \mathcal{Y}^\star$ be an arbitrary randomized mapping. Then $\mathcal{F} \circ \mathcal{M} : \mathcal{X} \to \mathcal{Y}^\star$ satisfies $\epsilon$-LDP, where $\circ$ denotes the composition of $\mathcal{F}$ and $\mathcal{M}$.*

### B. Perturbation Mechanisms

**Square-wave (SW) mechanism** [38] is the state-of-the-art LDP mechanism for distribution estimation over numerical data. Given a privacy budget $\epsilon$, SW satisfies $\epsilon$-LDP by taking an input $x \in [0, 1]$ and returning an output $y \in [-b, b+1]$ according to the following probability distribution: $\Pr[\mathcal{M}(x) = y] = \frac{e^\epsilon}{2be^\epsilon+1}$ if $|x - y| \leq b$, and $\Pr[\mathcal{M}(x) = y] = \frac{1}{2be^\epsilon+1}$ otherwise, where $b = \frac{\epsilon e^\epsilon - e^\epsilon + 1}{2e^\epsilon(e^\epsilon - 1 - \epsilon)}$. The output range will have a higher concentration around the input value as the privacy budget increases, which is beneficial for utility enhancement. SW can produce two estimated noisy histograms, post-processed by Expectation Maximization (EM) or Expectation Maximization with Smoothing (EMS) [38].

**Optimized unary encoding (OUE)** [11] is a state-of-the-art mechanism for frequency estimation over categorical data. Given a privacy budget $\epsilon$, OUE satisfies $\epsilon$-LDP by encoding an input $x \in \mathcal{X}$ as a $|\mathcal{X}|$-bit one-hot binary vector $B$, where only the $x$-th bit is set to 1. It then perturbs each bit of $B$, and outputs $B'$ by: $\Pr[B'[x] = 1] = p^1 = \frac{1}{2}$ if $B[x] = 1$, and $\Pr[B'[x] = 1] = p^0 = \frac{1}{e^\epsilon+1}$ if $B[x] = 0$. Let $count_x$ denote the number of inputs where $B[x] = 1$, and $n$ be the total user number. The frequency of $x$ is estimated by $f_x^{\text{OUE}} = \frac{count_x/n - p^0}{p^1 - p^0}$, and the variance [11] is $Var[f_x^{\text{OUE}}] = \frac{4e^\epsilon}{(e^\epsilon-1)^2 \cdot n}$.

### C. Range Query under LDP

The hierarchical tree has emerged as the most efficient data structure for answering 1-dimensional (a.k.a., 1-dim) range queries under LDP. Unlike straightforward methods that require summing up frequencies over all queried bins of a histogram, hierarchical trees recursively partition the entire domain into disjoint subdomains (a.k.a., intervals) to construct a $B$-ary tree. In this tree, each node indicates a coarse-grained interval and stores an estimated frequency. As such, a range query can be answered by summing up a few coarse-grained node frequencies (or parts of them), effectively reducing accumulated noise.

Directly extending hierarchical trees to $\lambda$-dimensional (a.k.a., $\lambda$-dim, where $\lambda > 1$) settings is impractical due to the excessive tree layers and exponentially increasing dimension combinations, which amplify noise. As a result, grid-based approaches are commonly used: given $m$ attributes, they estimate $m$ 1-D grids and $\binom{m}{2}$ 2-D grids, and derive higher-dimensional grids via maximum entropy [39] or weighted updating [7]. Existing methods further improve accuracy by optimizing granularity [7], using adaptive partitioning [3], or leveraging prefix-sum cubes [8]. Nevertheless, they all rely on a uniform or linear assumption for internal distribution.

### D. Cubic Spline Function

The cubic spline is extensively used in computer graphics and geometric modeling to obtain curves or motion trajectories that pass through specified points of the plane or 3-D space [40], [41], [42], [43]. Consider a Cartesian coordinate system with an x-axis and a y-axis. To interpolate the internal distribution over a target range $[d_l, d_r)$ on the x-axis, the CS function is defined as follows:

$$\begin{aligned} CS(t) = H_{00}(t) \cdot G[d_l] + H_{10}(t) \cdot (d_r - d_l) \cdot \nabla d_l + \\ H_{01}(t) \cdot G[d_r] + H_{11}(t) \cdot (d_r - d_l) \cdot \nabla d_r, \end{aligned} \tag{1}$$

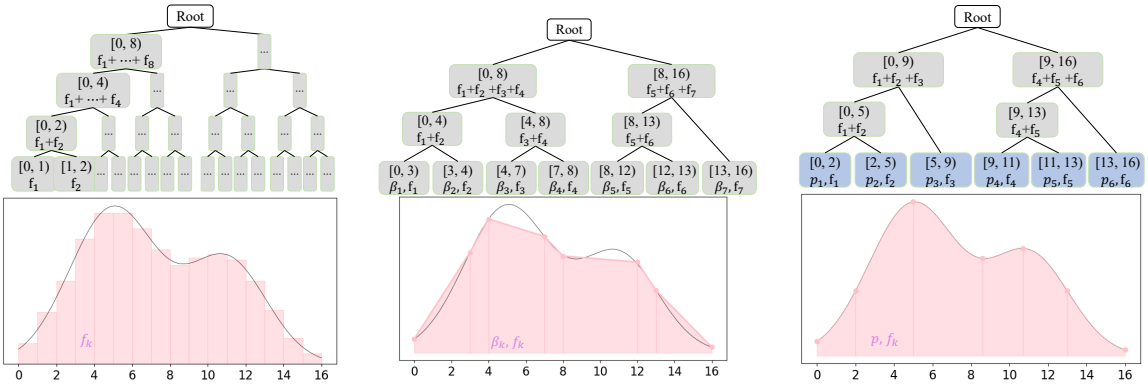
where $t = (c - d_l)/(d_r - d_l)$ and $c \in [d_l, d_r)$. Here, $G[d_l]$ and $G[d_r]$ denote the function values at $d_l$ and $d_r$, and $\nabla d_l$, $\nabla d_r$ denote their slopes. The functions $H_{00}(t) = 2t^3 - 3t^2 + 1$, $H_{10}(t) = t^3 - 2t^2 + 1$, $H_{01}(t) = -2t^3 + 3t^2$, $H_{11}(t) = t^3 - t^2$ are Hermite basis functions. Compared to other interpolation methods, each approximated interval curve is constructed from two selected adjacent breakpoints and their parameters, independent of other intervals, and can provide a more flexible and accurate approximation by selecting proper parameters. By using CS, we can also answer the same query with fewer nodes. For example, in Figure 2 (c), given a domain $[0, 16)$, a binary CS tree can answer a query $[0, 7]$ by summing only two node frequencies corresponding to $[0, 5)$ and $[5, 7)$, the latter is approximated by CS from the node of interval $[5, 9)$.

## IV. Problem Statement and Challenges

### A. Problem Statement

We consider a federated setting that involves an untrusted central server and a set of untrusted parties $\mathcal{P} = \{P_1, P_2, ...\}$. Each party $P_i \in \mathcal{P}$ has a fixed user set $U_i$, and each user belongs to exactly one party. Users communicate only with their registered party, motivated by cross-region data-sharing restrictions (e.g., in Europe). Each user $u_j \in U_i$ holds an ordinal record with $m$ attributes $\{A^1, \ldots, A^m\}$, denoted as $x_j = \langle x_j^1, \ldots, x_j^m \rangle$, where $x_j^\tau \in \mathcal{X}^\tau$ is the value of attribute $A^\tau$. As parties are untrusted, users will locally perturb their private records via LDP mechanisms to achieve $\epsilon$-LDP and upload only the noisy data to their parties.

Let $\Gamma$ be a subset of all attributes, i.e., $\Gamma \subseteq \{A^1, A^2, \cdots, A^m\}$, and $|\Gamma| = \lambda$. Our objective is to enable the central server to answer *$\lambda$-dimensional ($\lambda$-dim) federated range queries* over the data distributed across all parties $\mathcal{P}$, while satisfying $\epsilon$-LDP. Formally,




(a) HT with Uniform Assumptions (b) HT with Linear Assumptions (c) CS tree

Fig. 2: An illustration of hierarchical trees (HTs) with different assumptions (top) and their approximated data distributions (bottom). The black curve represents the ground truth distribution.

**Definition 2** ($\lambda$-Dim Federated Range Query)**.** *Let* $Q \triangleq \cap_{\tau=1}^{\lambda}[l_q^\tau, r_q^\tau]$ *represent the range over* $\lambda$ *dimensions, where* $l_q^\tau, r_q^\tau \in \mathcal{X}^\tau$ *and* $l_q^\tau < r_q^\tau$*. The* $\lambda$*-dim federated range query* $R_Q$ *returns the frequency of records whose queried attribute values fall within the specified* $\lambda$ *ranges, i.e.,*

$$R_Q = \frac{\sum_{i=1}^{|\mathcal{P}|}\sum_{j=1}^{|U_i|}\mathbb{I}_{\cap_{\tau=1}^{\lambda}\{l_q^\tau \le x_j^\tau \le r_q^\tau\}}}{\sum_{i=1}^{|\mathcal{P}|}|U_i|},$$

*where* $\mathbb{I}_y$ *denotes the indicator function which returns 1 if the predicate* $y$ *is true and 0 otherwise.*

**Remark 1.** *Intuitively, a naive solution is to collect perturbed data directly from users of different parties, based on which the central server can answer the federated range queries. However, this approach may incur* ***excessive communication costs****. In practice, both the user population and the item domain are typically very large. For example, consider a scenario with* $5,000,000$ *users and a domain size of* $|\mathcal{X}| = 2,000$*. The state-of-the-art frequency estimation mechanism OUE [11] needs to encode each input value into a* $2,000$*-bit vector before sending the perturbed version to the server. This results in a communication cost of approximately* $1 \times 10^{10}$ *bits, which is prohibitively high and impractical for many real-world applications. Although more communication-efficient protocols, such as OLH, can be used, their communication overheads remain substantial (i.e.,* $O(|\mathcal{U}|)$*). Additionally, decoding the randomized data from these mechanisms at the server requires an exhaustive scan of the entire domain* $\mathcal{X}$ *for each user, which is* ***computationally infeasible*** *for large domains [14], [15], [16]. Furthermore,* ***such flat methods would introduce significant accumulated noise errors****, making the results unreliable for use [6].*

### *B. Straw-Man Solution and Challenges*

To answer $\lambda$-dim federated range queries, a straightforward solution is to apply an existing range query mechanism (e.g., PrivNUD [2]) in each party. Each party uploads its constructed hierarchical structure to the server, who then queries each received local structure and sums up all the local answers to derive the overall frequencies. However, this solution overlooks the non-IID problem, posing the following significant challenges in answering federated range queries accurately.

**Large approximation errors under unrealistic assumptions.** Data heterogeneity results in varying frequency distributions across parties. However, all existing LDP solutions assume that the data within each node of the hierarchical structures follows either a uniform or linear distribution. This mismatch can lead to large approximation errors. For example, in Figure 2(b), the answer of a query $[5, 10]$ is calculated by summing the frequencies of interval $[7, 8)$ and the partial frequencies $[5, 7)$ and $[8, 10)$ under linear assumptions. That is, they first estimate the integral ratios of intervals $[5, 7)$ to $[4, 7)$, and $[8, 10)$ to $[8, 12)$ by assuming linear internal distributions within these intervals. Then, based on these ratios, they approximate the answers accordingly. Such a linear assumption often introduces significant approximation errors (e.g., the large gap within $[4, 7)$ in Figure 2(b)), as data distributions rarely conform exactly to linearity in practice. Although we can apply a finer granularity to leaf nodes, it will require more noisy nodes to answer the same query, leading to larger noise.

**Complex server-side aggregation under the data heterogeneity and LDP noise.** The non-IID nature of data complicates frequency estimation and aggregation across parties. Concretely, local frequencies [1] of certain ranges estimated by some parties may not contribute positively to estimating the global ones. For instance, certain items may be frequently purchased in Amazon Asia but remain less popular in Amazon Europe. The frequencies of such items in certain parties do not offer valuable information in aggregating the overall frequency, since they may be easily overwhelmed by LDP noise. In the following Section V, we explore these issues and address them by introducing our proposed mechanism PrivCS,

[1]For ease of understanding, we refer to party-level frequencies estimated in a single party $P_i$ as local frequencies.

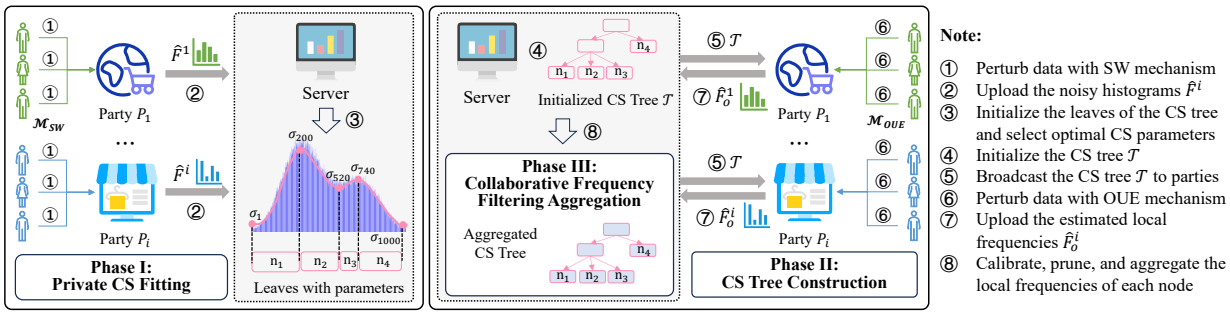



Fig. 3: An overview of PrivCS.

which effectively mitigates these issues.

## V. PrivCS: A Private Cubic Spline Mechanism

To address the challenges as aforementioned, we propose private cubic spline (PrivCS), which leverages a few cubic splines to capture the underlying data distribution and devises a unified hierarchical tree structure to support federated range queries by integrating the fitting results.

### A. Design Rationale

The cubic spline has a nice property of accurately approximating skewed distributions as curves via just a few parameters. As the building block to achieve accurate range queries, the approximated distribution inherently eliminates the uniform or a linear assumption, as made in the existing work [2], [3], [5]. Furthermore, CS is capable of preventing the Runge phenomenon compared to high-degree interpolation, while providing more accurate approximations than low-degree methods [17]. This inspires us to design PrivCS that fits the data distribution via cubic splines for answering range queries over arbitrary distributions. On the other hand, since the data heterogeneity may result in varying party-level distributions, fitting them independently fails to capture the overall distribution geometry. Therefore, to address this issue, PrivCS first fits the overall distribution and then constructs a globally optimal hierarchical tree (i.e., CS tree). Each leaf node represents an interval and stores the corresponding CS parameters and interval frequency. Non-leaf nodes represent their intervals by combining those of their children and store only the frequencies. In contrast to other hierarchical trees [2], [3], [5], CS tree enables us to use fewer nodes (thus less LDP noise [3]) and more accurate approximations to model the skewed distribution, making it well-suited for federated settings.

Moreover, some party-level frequency estimates can be easily overwhelmed by LDP noise. Simply aggregating such estimates of the same interval to compute its overall frequency may introduce excessive noise in federated settings. To this end, we introduce the collaborative frequency filtering (CF) aggregation. In this aggregation, we leverage some valuable frequency information across parties for the same intervals to adaptively calibrate the frequency estimates for each node, and selectively prune the local frequencies of certain nodes that exceed the pruning threshold by analyzing the pruning error, thereby reducing the cumulative LDP noise.

### B. An Overview of PrivCS

The above design rationale leads to our private cubic spline (PrivCS) mechanism. Figure 3 shows its overview, which consists of three phases, namely private CS fitting, CS tree construction, and collaborative frequency filtering aggregation.

**Phase I: Private CS fitting.** Our goal is to produce the optimal CS fitting using a set of curves tailored to model interval-wise distributions. Each curve models the internal distribution of an interval, which includes all the bucketized values between the starting and ending bins (a.k.a *breakpoints*). For example, the interval $[2, 5)$ includes $\{2, 3, 4\}$ between breakpoints 2 and 5. Eventually, the fitting should capture the entire distribution geometry by selecting the optimal parameters for each interval. Thus, to gain fundamental insight into the invisible distribution, we first estimate a global histogram under LDP as a reference.

Specifically, in step ①, each party samples an $\alpha$ fraction of its users and applies the SW mechanism $\mathcal{M}_{\mathrm{SW}}(\cdot)$ with privacy budget $\epsilon$ to obtain a noisy histogram $\hat{F}^i$. In step ②, parties upload these histograms to the server for aggregation. Based on the aggregated histogram, the server (step ③) determines breakpoints to partition the domain into disjoint intervals (initializing the leaves of the CS tree), and then selects the optimal CS parameters by minimizing the expected approximation error within each leaf's internal distribution.

**Phase II: CS tree construction.** We construct the CS tree in this phase. The server first initializes the tree from bottom to top based on the estimated leaves, and broadcasts the tree with necessary parameters to each party for further estimation (steps ④–⑤). Upon receiving them, each party divides the remaining users into several groups (depending on the tree structure) to estimate frequencies of different intervals. Users from each group in Party $P_i$ perturb their data using OUE $\mathcal{M}_{OUE}(\cdot)$ and upload to $P_i$ (step ⑥). Party $P_i$ then aggregates the sanitized data from these groups to estimate the frequencies of the corresponding nodes (i.e., intervals) in the CS tree. Finally, the parties transmit their estimated frequencies back to the server for aggregation (step ⑦).

**Algorithm 1:** Private CS Fitting

**Input:** Parties $\mathcal{P}$, dividing ratio $\alpha$, and privacy budget $\epsilon$
**Output:** The sorted breakpoint set $\Delta^R$ and corresponding parameters

```
// Party side
1 for P_i ∈ P do
2     P_i requires a small part of its users α · U^i to estimate the noisy
        histogram F̂^i via SW with privacy budget ε;
3     P_i uploads F̂^i to server;
// Server side
4 The server aggregates histograms received from each party, then
    applies EMS and EM post-processing steps to obtain the overall
    histograms F̂_s and F̂_m;
5 The server solves Equation (3) to obtain the breakpoint set Δ;
6 Sort breakpoints in Δ in an ascending order by the bin index as Δ^R;
7 for 1 ≤ idx ≤ |Δ^R| − 1 do
8     σ_l = Δ^R[idx] and σ_r = Δ^R[idx + 1];
9     Solve Equation (4) with σ_l and σ_r to obtain the optimal
        parameters of each leaf node;
10 return Δ^R and corresponding parameters;
```

**Phase III: Collaborative frequency filtering aggregation.** The proposed CF aggregation (step ⑧) is applied colloboratively to *calibrate*, *prune*, and *aggregate* the local frequencies of each node. For each node's interval, we have two frequency estimates: one from the histogram estimation phase (at the beginning of Phase I), and the other from the frequency estimation phase (Phase II). These two estimates are intrinsically related with each other, as they target the same interval frequency via different methods.

The key idea is to leverage these relationships to reduce cumulative errors. Therefore, the CF strategy adaptively calibrates the local frequency estimates for each node while selectively pruning some estimates before aggregation. Finally, the server applies a post-processing step to further enhance overall frequency consistency, thus producing an optimal CS tree.

**Answering 1-dim federated range queries.** Given a federated range query $Q = [l_q, r_q]$, PrivCS answers by summing (i) the frequencies of the *maximal fully covered nodes*—those whose intervals lie inside $[l_q, r_q]$ while their parents do not—and (ii) partial frequencies from the two boundary leaves that partially overlap $[l_q, r_q]$. For instance, given the CS tree shown in Figure 2(c), the answer to the query $[4, 10]$ is calculated by summing up the frequency of the interval $[5, 9)$, a partial frequency of the interval $[2, 5)$, and a partial frequency of the interval $[9, 11)$. The proportions for intervals $[2, 5)$ and $[9, 11)$ are computed based on the ratios of the overlap integrals of the corresponding nodes; specifically, $\pi_2 = \int_4^5 CS_2(t)dt / \int_2^5 CS_2(t)dt$ and $\pi_4 = \int_9^{10} CS_4(t)dt / \int_9^{11} CS_4(t)dt$, where $CS_2(\cdot)$ and $CS_4(\cdot)$ represent the corresponding CS functions with their parameters. Thus, the final answer to $Q$ is computed as $\pi_2 \hat{f}_2 + \hat{f}_3 + \pi_4 \hat{f}_4$, where $\hat{f}_k$ denotes the noisy frequency of node $n_k$.

### C. Private CS Fitting

The objective of this phase is to obtain a set of breakpoints to produce disjoint intervals and fix the optimal interval-wise CS parameters. They will then serve as the foundation for constructing the CS tree in the subsequent phase. As such, the private CS fitting includes three steps: (1) the overall histogram estimation, (2) the breakpoints selection, and (3) the CS parameter optimization. The first step is to provide a rough reference for the subsequent breakpoints and parameters selections, which are the main challenges in this phase. As illustrated in Figure 2(c), the breakpoint set $\Delta = \{0, 2, 5, 9, 11, 13, 16\}$ determines six intervals: $[0, 2), [2, 5) \ldots, [13, 16)$. The CS fitting characterizes each interval by a pair of neighboring *breakpoints*, along with their associated frequencies and derivatives as initial parameters. For each interval $[l, r)$, we will then optimize the CS function in the parameter selection step by modifying Equation (1) as follows:

$$\begin{aligned} CS(t) = H_{00}(t) \cdot \hat{F}_s[l] + H_{10}(t) \cdot (r - l) \cdot \nabla\sigma_l + \\ H_{01}(t) \cdot \hat{F}_s[r] + H_{11}(t) \cdot (r - l) \cdot \nabla\sigma_r, \end{aligned} \tag{2}$$

where $t = (c - l)/(r - l)$ and $c \in [l, r)$, $\hat{F}_s[l]$ and $\nabla\sigma_l$ (resp. $\hat{F}_s[r]$ and $\nabla\sigma_r$) represent the noisy frequency and the noisy first derivative at $l$-th bin $\sigma_l$ (resp. $r$-th bin $\sigma_r$).

However, the breakpoints selection is non-trivial, as it requires balancing the accurate approximation of the distribution's geometry with the accumulated LDP noise. Furthermore, directly optimizing the CS parameters based on the estimated data distribution can introduce excessive noise, particularly when the privacy budget is constrained. We will address these challenges as follows.

**Histogram estimation.** To obtain the global histogram across all parties, as outlined in lines 1–3 of Algorithm 1, each party $P_i \in \mathcal{P}$ uploads its noisy histogram $\hat{F}^i$ estimated by SW [38] to the server. The server then aggregates these histograms and applies the EM and EMS optimization steps of SW to obtain $\hat{F}_m$ and $\hat{F}_s$ (line 4).

**Breakpoints selection.** The breakpoints define the interval partition. Increasing the number of breakpoints yields finer-grained approximations of the underlying data distribution but also results in more leaves (thus more layers), which introduce more LDP noise [2]. To strike a balance between capturing the distribution's geometry and minimizing noise accumulation, we heuristically select breakpoints based on the approximated first and second derivatives of the bins, leveraging the observation that bins with near-zero derivatives exhibit greater stability under LDP noise. More precisely, let $\sigma_j$ denote the $j$-th bin of the estimated histogram $\hat{F}_s$ for the attribute domain $\mathcal{X}_\tau$, the approximated first and second derivatives at $\sigma_j$ are calculated as follows: $\nabla\sigma_j = \frac{\hat{F}_s[j+1] - \hat{F}_s[j-1]}{2}$ and $\nabla^2\sigma_j = \hat{F}_s[j-1] - 2\hat{F}_s[j] + \hat{F}_s[j+1]$. The breakpoint set $\Delta$ comprises all bins where both derivatives vanish within a $\delta$-tolerance:

$$\Delta = \{\sigma_j | \nabla\sigma_j \in [-\delta, \delta], \nabla^2\sigma_j \in [-\delta, \delta], \sigma_j \in \mathcal{X}_\tau\}. \tag{3}$$

Note that $\delta$ could be set as a small positive value inversely proportional to the total user population $|U| = \sum_{i=1}^{|\mathcal{P}|} |U^i|$. For the first and last bins, we use backward and forward differences instead. The server sorts $\Delta$ in ascending order. Each interval is defined by a pair of adjacent breakpoints in this ordered set.

**Algorithm 2:** CS tree Construction

**Input:** Parties $\mathcal{P}$, dividing ratio $\alpha$, breakpoint set $\Delta^R$, and privacy budget $\epsilon$

**Output:** All the estimated frequencies $\{\hat{F}_o^1, \hat{F}_o^2, \cdots, \hat{F}_o^{|\mathcal{P}|}\}$

```
// Server side: CS tree initialization
```

1 Construct a balanced binary tree $\mathcal{T}$ based on $\Delta^R$;
2 **for** *node $n_k$ in postorder traversal* **do**
3 Compute $Err$ and $Err'$ for $\mathcal{T}$ with $n_k$ and w/o $n_k$ via Equation (5);
4 **if** $Err > Err'$ **then**
5 Remove $n_k$;
6 The server broadcasts the reduced $\mathcal{T}$ to each party;

```
// Party side: User allocation and node frequency estimation
```

7 **for** $P_i \in \mathcal{P}$ **do**
8 $P_i$ assigns available users $U_0^i = U^i - \alpha \cdot U^i$ to the root $n_0$;
9 **for** *each node $n_k$ of $\mathcal{T}$ in level order traversal* **do**
10 **if** *$n_k$ is root* **then**
11 Set $U_k^{i'} = \emptyset$;
12 **else**
13 Compute $h_k$, the height of the subtree rooted at $n_k$;
14 Sample $\lceil |U_k^i|/h_k \rceil$ users from $U_k$ as $U_k^{i'}$;
15 Allocate $U_k^{i'}$ to $n_k$ for frequency estimation;
16 **for** $n_c \in children(n_k)$ **do**
17 Assign $U_k^i - U_k^{i'}$ to $n_c$;
18 **for** *each user $u_j \in U_0^i$* **do**
19 Assign intervals of nodes $\mathcal{N}_i = \{n_k | u_j \in U_k^{i'}\}$ to $u_j$;
20 $u_j$ uploads the OUE-perturbed value (with $\epsilon$) to $P_i$;
21 $P_i$ aggregates each node's frequency and uploads the estimated frequencies $\hat{F}_o^i$ to the server.
22 **return** All the estimated frequencies $\{\hat{F}_o^1, \hat{F}_o^2, \cdots, \hat{F}_o^{|\mathcal{P}|}\}$;

**Parameters selection.** Now we explain how the server selects the optimal parameters for a given leaf's interval $[l, r)$. Recall that in Equation (2), the optimal CS function $CS^*(\cdot)$ for $[l, r)$ depends on four key parameters, i.e., frequencies and the first derivatives at the bins $\sigma_l$ and $\sigma_r$. While one could directly use the noisy estimates $\hat{F}_s[l]$, $\hat{F}_s[r]$, $\nabla\sigma_l$, and $\nabla\sigma_r$ obtained from the noisy histogram $\hat{F}_s$, this approach would propagate significant LDP noise into the final results. To achieve a balance between approximation error and computational efficiency, we choose to refine only the first derivatives $\nabla\sigma_l$ and $\nabla\sigma_r$, while retaining the noisy frequency estimates $\hat{F}_s[l]$ and $\hat{F}_s[r]$. This choice is supported by empirical observation showing that frequencies exhibit greater robustness to noise perturbations compared to the first derivative.

In a nutshell, our objective is to determine the optimal $\nabla\sigma_l^*$ and $\nabla\sigma_r^*$ that minimize the expected approximation error for all possible range queries $Q_\cap \triangleq \{[l_q, r_q] | [l, r) \cap [l_q, r_q] \neq \emptyset\}$ intersecting the interval $[l, r)$, as the CS function (or uniform/linear assumption) only works when intersecting. Note that $Q_\cap$ are derived by enumerating all the possibly intersected queries with uniform arrival probabilities [2]. Formally, we aim to select

$$\nabla\sigma_l^*, \nabla\sigma_r^* = \underset{\nabla\sigma_l, \nabla\sigma_r}{\arg\min} \frac{1}{|Q_\cap|} \sum_{[l_q, r_q] \in Q_\cap} \sum_{\hat{F} \in \{\hat{F}_s, \hat{F}_m\}} \eta(l, r, l_q, r_q, CS(\cdot), \hat{F}), \tag{4}$$

to obtain $$CS^*(t) = H_{00}(t) \cdot \hat{F}_s[l] + H_{10}(t) \cdot (r - l) \cdot \nabla\sigma_l^* + H_{01}(t) \cdot \hat{F}_s[r] + H_{11}(t) \cdot (r - l) \cdot \nabla\sigma_r^*,$$

where $\eta(\cdot)$ denotes the error function. Note that the approximation error is manifested in answering the partial frequency of the intersected subinterval $[l_q, r_q] \cap [l, r)$, as this frequency is computed by multiplying the *frequency proportion* (approximated by the CS function or other assumptions) with the interval frequency of $[l, r)$. For example, given a node $n_k$ with the interval $[2, 5)$, the partial frequency of $[4, 5)$ is $\pi_k f_k$, where $\pi_k$ is the frequency proportion of $[4, 5)$ and $f_k$ is the interval frequency of $n_k$. Therefore, we define $\eta(\cdot)$ to measure the difference between the frequency proportions of $[l_q, r_q] \cap [l, r)$ computed by $CS(\cdot)$ (with parameters $\nabla\sigma_l$ and $\nabla\sigma_r$) and that estimated from the reference histogram, i.e.,

$$\eta(l, r, l_q, r_q, CS(\cdot), \hat{F}) = [\varphi(l, r, l_q, r_q, CS(\cdot)) - \omega(l, r, l_q, r_q, \hat{F})]^2,$$
$$\varphi(l, r, l_q, r_q, CS(\cdot)) = \frac{\int_{l_\cap}^{r_\cap} CS(t)dt}{\int_l^r CS(t)dt}, \omega(l, r, l_q, r_q, \hat{F}) = \frac{\sum_{t=l_\cap}^{r_\cap} \hat{F}[t]}{\sum_{t=l}^{r} \hat{F}[t]},$$

where $l_\cap = \max\{l_q, l\}$ and $r_\cap = \min\{r_q, r\}$. Here $\varphi(\cdot)$ computes the definite integral of the CS function $CS(\cdot)$, whereas $\omega(\cdot)$ computes the discrete sum of the histogram $\hat{F}$ over the corresponding bins. To accelerate the optimization of Equation (4), we search $\nabla\sigma_l$ (resp. $\nabla\sigma_r$) in the range of $[\nabla\sigma_l - e^{-\epsilon}/2, \nabla\sigma_l + e^{-\epsilon}/2]$ (resp. $[\nabla\sigma_r - e^{-\epsilon}/2, \nabla\sigma_r + e^{-\epsilon}/2]$), where the term $e^{-\epsilon}/2$ is introduced to narrow down the searching space as $\epsilon$ increases, since a larger $\epsilon$ results in more accurate estimations, and vice versa. Finally, the server searches for the optimal parameters by solving Equation (4) for each interval defined by the breakpoints in $\Delta$. These intervals, along with their corresponding optimal parameters, are stored as the leaves of the CS tree (lines 5–9 of Algorithm 1).

### D. CS Tree Construction

Given the leaves with optimal parameters, the next step is to construct the CS tree, which involves three steps: CS tree initialization, user allocation, and frequency estimation.

**CS tree initialization.** At first, the server constructs a default binary tree from bottom to top using these leaves (line 1 in Algorithm 2). Then, the server adaptively reduces non-leaf nodes. The reduction of a node involves deleting it and linking its all child nodes to its parent node. Such a reduction occurs if the average error of deleting a node is smaller than the error from preserving it [2], [3]. Specifically, let $\mathcal{X}_\tau$ denote the domain of attribute $\tau$. We evaluate the average error by calculating the expected accumulated noise from nodes intersected by all possible query ranges,

$$\sum_{n_k \in \mathcal{N}} w_k \cdot Err(\hat{f}_k), \quad w_k = \frac{l_k(|\mathcal{X}_\tau| - r_k + 1) - l_p(|\mathcal{X}_\tau| - r_p + 1)}{(|\mathcal{X}_\tau| + 1)|\mathcal{X}_\tau|/2}, \tag{5}$$

where $[l_k, r_k]$ (resp. $[l_p, r_p]$) is the interval stored of node $n_k$ (resp. its parent node $n_p$) and $w_k$ denotes the probability of $n_k$ being involved in queries. Since we will use OUE for

frequency estimation, whose variance is inversely proportional to the allocated user population, we approximate $Err(\hat{f}_k)$ by $1/\alpha_k$ for convenience, where $\alpha_k$ is the user allocation ratio for $n_k$ [2], [3]. Once the reduction is completed, the server broadcasts the initialized CS tree to each party for user allocation (lines 2–6 of Algorithm 2).

**User allocation.** Following the previous study [3], each party $P^i$ distributes its users along relative paths in the CS tree. All users are initially assigned to the root node, which serves as the starting point and does not require frequency estimation. For each non-root node $n_k$, it inherits the set of $\lceil |U_k^i|/h_k \rceil$ users allocated to its parent node, uniformly samples a subset $U_k^{i'}$ from it for itself, and passes the remaining users to its children. Here, $|U_k^i|$ represents the number of users remaining after user allocation to the parent node of $n_k$, and $h_k$ denotes the height of the sub-tree rooted at $n_k$. For the leaf nodes, they are all assigned $U_k^i \setminus U_k^{i'}$ users.

**Frequency estimation.** Each party requires its users to perturb and upload their sanitized data, to estimate frequencies of all nodes by OUE with privacy budget $\epsilon$. Note that each user only participates in the assigned nodes' frequency estimations. Then, each party sends all the nodes' local frequencies to the server for aggregation (lines 7–21 of Algorithm 2).

### *E. Collaborative Frequency Filtering Aggregation*

The most straightforward approach for estimating the overall interval frequency of a node is to aggregate the noisy local frequencies reported by all parties. However, this method may introduce significant noise in cross-party settings due to two key factors as follows:

- A small privacy budget $\epsilon$ potentially leads to some extremely large or small local frequency estimations.
- Cross-party heterogeneity makes certain local frequencies particularly susceptible to being overwhelmed by LDP noises, especially when these parties have only a small share of specific intervals.

To overcome these issues, we propose *collaborative frequency filtering (CF) aggregation* consisting of three key steps: 1) local frequency calibration, 2) local frequency pruning, and 3) local frequency aggregation. The CF method operates on all node-level frequency estimates within the CS tree.

**Local frequency calibration.** To mitigate each node's aggregation error caused by extreme party-level frequency estimates, this step calibrates each party's Phase II frequency estimate using Phase I frequency references. The intuition is that, although the $P_i$-level and the overall frequency estimates in Phase I are noisy, their relative relation can serve as a prior reference. Consider an arbitrary node with interval $[l, r)$ in the CS tree, its perturbed local frequency estimated by party $P_i$ via OUE in Phase II is denoted as $\hat{f}_o^i$. For the same node, we can also obtain its local frequency $\hat{f}_s^i = \sum_{j\in[l,r)} \hat{F}_s{}^i[j]$ and the overall frequency $\hat{f}_s = \sum_{j\in[l,r)} \hat{F}_s[j]$ estimated by SW in Phase I. Therefore, instead of using the unstable frequencies themselves, we leverage the relative deviation between $\hat{f}_s^i$ and $\hat{f}_s$ to constrain and refine the OUE estimates as follows:

**Algorithm 3:** CF Aggregation

**Input:** Parties $\mathcal{P}$, overall histogram $\hat{F}_s$ estimated by SW, all the estimated frequencies $\{\hat{F}_o^1, \hat{F}_o^2, \cdots, \hat{F}_o^{|\mathcal{P}|}\}$, and the overall CS tree $\mathcal{T}$
**Output:** The optimal CS tree $\mathcal{T}$
`// Server side`
1 The server calculates the optimal threshold $\xi^*$ with corresponding $\xi_1^*$ and $\xi_2^*$ via Equation (9);
2 **for** *each node $n$ of $\mathcal{T}$ in post-order traversal* **do**
3 Calculate the overall interval frequency $\hat{f}_s$ of node $n$ estimated in Phase I by summing its corresponding bins' frequencies in $\hat{F}_s$ up;
4 **for** $1 \le i \le |\mathcal{P}|$ **do**
5 Calibrate each $\hat{f}_o^i$ of node $n$ estimated by $P_i$ by Equation (6), where $\hat{f}_o^i$ is the OUE-estimated local frequency of node $n$ in $\hat{F}_o^i$;
6 **if** $\xi^* > 0$ *and* $\hat{f}_o^i/\hat{f}_s \le \xi_1^*$ *and* $\hat{f}_s \le \xi_2^*$ *and* $\hat{f}_o^i \le \xi^*$ **then**
7 $\hat{f}_o^i = 0$;
8 Compute the overall OUE-estimated frequency $\hat{f}_o^i = \sum_{1\le i\le|\mathcal{P}|} |U^i| \cdot \hat{f}_o^i/|U|$, where $|U|$ (resp. $|U^i|$) denotes the overall (resp. $i$-th party's) user population;
9 Employ the weighted averaging and frequency consistency procedures for $\mathcal{T}$ to enhance the utility and consistency;
10 **return** $\mathcal{T}$;

$$\hat{f}_o^i = \hat{f}_o^i \cdot (1+\mu), \mu = \begin{cases} -\frac{\hat{f}_o^i - b}{e^\epsilon \hat{f}_o^i} & \text{if } \hat{f}_o^i > b, \\ \frac{a - \hat{f}_o^i}{e^\epsilon \hat{f}_o^i} & \text{if } \hat{f}_o^i < a, \end{cases} \tag{6}$$

where $a = \hat{f}_s^i \cdot (1-\beta), b = \hat{f}_s^i \cdot (1+\beta), \beta = \frac{|\hat{f}_s^i - \hat{f}_s|}{\hat{f}_s}$.

The above refinement calibrates estimated local frequencies that fall outside the feasible range $[a, b]$, which is derived from the offset $\beta$ between $\hat{f}_s^i$ and $\hat{f}_s$. The $\beta$ acts as a frequency-based proxy for heterogeneity between the $P_i$'s distribution and the overall distribution on this interval. That is, for $\hat{f}_o^i$ that falls outside this range, we treat it as infeasible and refine it, as it exceeds the bound $[a, b]$ under the current heterogeneity. Therefore, we use the $\mu$ to adjust the infeasible frequencies to be closer to $[a, b]$. The $\mu$ consists of the difference ratio between $b - \hat{f}_o^i$ (resp. $a - \hat{f}_o^i$) and $\hat{f}_o^i$ when $\hat{f}_o^i > b$ (resp. $\hat{f}_o^i < a$) and the decay factor $e^{-\epsilon}$, which weakens $\mu$ as $\epsilon$ increases, since the OUE estimates become more accurate.

**Local frequency pruning.** After calibrating node-level frequencies (lines 5–7 of Algorithm 3), we must decide whether to *accept* or *prune* the party-level estimates during global aggregation. This decision presents a key trade-off: acceptance introduces additional LDP noise, while rejection may result in the loss of meaningful frequency information. We therefore prune low local frequencies by analytically selecting an optimal *pruning threshold* $\xi^*$ that minimizes the expected rejection error and yields lower error than acceptance.

***Errors of acceptance and rejection.*** For a node with interval $[l, r)$ and a party $P_i$, denote the true local frequency by $f_o^i$ and its Phase II estimate by $\hat{f}_o^i$. Given a pruning threshold $\xi$, we measure the acceptance loss as follows:

$$\kappa_{\text{acc}}(\hat{f}_o^i, f_o^i; \xi) = (\xi - \hat{f}_o^i) \cdot \mathbb{I}\{f_o^i \geq \xi,\ \hat{f}_o^i \leq \xi\} + |\hat{f}_o^i - f_o^i| \cdot \mathbb{I}\{f_o^i < \xi,\ \hat{f}_o^i \leq \xi\},$$

which penalizes (i) *missed-above-threshold* events and (ii) *accepted-below-threshold* events. Here $\mathbb{I}\{\cdot\}$ denotes the indicator, which equals 1 if the event in braces holds and 0 otherwise. Then the expected error of acceptance is :

$$Err_1(\xi) = \mathbb{E}_{f_o^i} \mathbb{E}_{\hat{f}_o^i \mid f_o^i} \big[\kappa_{\text{acc}}(\hat{f}_o^i, f_o^i; \xi)\big]. \tag{7}$$

The probabilities inside Equation 7 are evaluated via the Gaussian CDF $\Phi(\frac{\hat{f}_o^i - f_o^i}{Var})$, where $Var$ denotes the variance of OUE, since a local frequency estimated by OUE can be approximated by a Gaussian distribution [14]. To satisfy LDP, the prior of $f_o^i$ is approximated from the Phase I overall histogram $\hat{F}_s$, treating it as the "true" frequency (implemented via an empirical sample set $\mathcal{V}$), and $\hat{f}_o^i$ is assumed as the worst-case "noisy" frequency appearing uniformly.

Similarly, if we *reject* (prune) estimates with $\hat{f}_o^i \leq \xi$, the contribution is set to zero. The rejection loss is then produced:

$$\kappa_{\text{rej}}(f_o^i; \xi) = \xi \cdot \mathbb{I}\{f_o^i \geq \xi,\ \hat{f}_o^i \leq \xi\} + f_o^i \cdot \mathbb{I}\{f_o^i < \xi,\ \hat{f}_o^i \leq \xi\}. \tag{8}$$

By substitute $\kappa_{\text{acc}}(\cdot)$ in Equation 7 with $\kappa_{\text{rej}}(\cdot)$, we can obtain the expected rejection error $Err_0(\cdot)$.

***Threshold selection.*** Small party-level frequencies with small global share are more likely to be overwhelmed by LDP noise. To reflect this, we factorize $\xi = \xi_1 \cdot \xi_2$, where $\xi_1$ encodes *local importance*, measured by the ratio between $\hat{f}_o^i$ and $\hat{f}_s$, and $\xi_2$ encodes *global importance* (the Phase I overall frequency on the node). This factorization reduces the search dimensionality and adapts to cross-party heterogeneity. We search over $\xi_1 \in [0, e^{-\epsilon}/2]$ and $\xi_2 \in [\xi_1, e^{-\epsilon}/2]$, which matches the typical OUE deviation scale. The optimal pruning threshold is then chosen as

$$\begin{aligned} \xi_1^*, \xi_2^* &= \arg\min_{\xi_1, \xi_2}\ Err_0(\xi) \\ \text{s.t.} \quad & Err_0(\xi) < Err_1(\xi), \quad \xi = \xi_1 \cdot \xi_2, \end{aligned} \tag{9}$$

and we set $\xi^* = \xi_1^* \cdot \xi_2^*$. If no feasible candidate satisfies the constraint, pruning is disabled (line 1 and lines 6–7 of Algorithm 3). In the implementation, we approximate $\mathbb{E}_{f_o^i}[\cdot]$ using $|\mathcal{V}| = 100$ draws from the empirical prior induced by $\hat{F}_s$ and compute probabilities with $\Phi(\cdot)$.

**Local frequency aggregation.** The CF enables us to refine and prune certain local frequencies by considering both local and global frequency information. Subsequently, we populate the overall CS tree with these calibrated frequencies. For each node, we compute and store the overall frequency. For instance, given a node's frequency $\hat{f}^0$ from party $P_0$ with $|U^0|$ users and $\hat{f}^1$ from party $P_1$ with $|U^1|$ users, the overall frequency of this node is $(\hat{f}^0 \cdot |U^0| + \hat{f}^1 \cdot |U^1|)/(|U^0| + |U^1|)$. However, frequency inconsistency issues may arise. For example, the inconsistencies between a parent node's frequency and the sum of its child nodes, as well as discrepancies between the frequencies of leaf nodes estimated by SW and OUE. To mitigate these issues, we employ weighted averaging and frequency consistency post-processing (line 9 of Algorithm 3), following the previous studies [2], [3]. We provide the details in Appendix A.

TABLE II: Communication costs on server-side.

| | PrivNUD or AHEAD | OUE | PriPL-Tree or PrivCS |
|---|---|---|---|
| Comm. | $O(b \cdot \lvert\mathcal{N}\rvert \cdot \lvert\mathcal{P}\rvert)$ | $O(\lvert\mathcal{X}\rvert \cdot \lvert\mathcal{U}\rvert)$ | $O(b \cdot \lvert\mathcal{P}\rvert \cdot (\lvert\mathcal{N}\rvert + \lvert\mathcal{X}\rvert))$ |

### *F. Privacy, Utility, and Communication Cost Analysis*

**Privacy analysis.**

In Theorem 2, we prove that the PrivCS mechanism satisfies $\epsilon$-LDP. Due to the page limit, we defer a detailed proof to Appendix B.

**Theorem 2.** *The PrivCS mechanism satisfies $\epsilon$-LDP.*

*Proof sketch.* In PrivCS, all users only upload the noisy data perturbed by an LDP mechanism (i.e., SW or OUE) to their parties once, either in the first or second phase. Other operations on both party-side and server-side are based on the sanitized data that enjoys the desired post-processing property of LDP. Thus, the PrivCS mechanism satisfies $\epsilon$-LDP. □

**Utility analysis.** We prove that the proposed CS tree leads to smaller errors than the linear-assumption method (i.e., PriPL-Tree) when modeling the same distribution in the single-party setting under the same target error. Due to the page limit, we defer a detailed proof to Appendix C.

**Theorem 3.** *Given a target uniform error $1 > \delta_u > 0$, let $\mathrm{Var}_{cs}$ (resp. $\mathrm{Var}_{lin}$) denote the variance of frequencies in the CS tree (resp. PriPL-Tree), we have: $\frac{\mathrm{Var}_{\text{cs}}}{\mathrm{Var}_{\text{lin}}} = \Theta\big(\delta_u^{1/4}\big)$.*

The result implies that, under the same fitting error requirement, the CS tree results in a lower error. Equivalently, to achieve the same variance, the linear-assumption method must produce much larger fitting errors than the CS tree.

**Communication costs on server-side.** Let $|\mathcal{N}|$ denote the total number of nodes in the hierarchical tree, $|\mathcal{X}|$ be the domain size of the attribute, $|\mathcal{P}|$ be the number of parties, and each (node/interval, frequency) pair consumes $b$ bits. In the cross-party setting, the main bottleneck is server-side communication (i.e., $O(|\mathcal{U}| \cdot |\mathcal{X}|)$ when using OUE directly), whereas user-to-party reporting ($|U|/|\mathcal{P}|$ per party) is unavoidable for accurate estimation; thus we focus on server-side cost. In PrivCS, each party uploads a noisy histogram and OUE estimates for all CS-tree nodes, costing $b \cdot |\mathcal{P}| \cdot |\mathcal{X}|$ and $b \cdot |\mathcal{P}| \cdot |\mathcal{N}|$, respectively. No extra communication is required for aggregation, yielding a total cost of $O\big(b \cdot |\mathcal{P}| \cdot (|\mathcal{X}| + |\mathcal{N}|)\big)$. For comparison, we adapt existing baselines by having each party upload aggregated user results to the server and assume all methods use the same tree structure (hence the same $|\mathcal{N}|$). All results are summarized in Table II. It is worth noting that while communication overheads are comparable, PrivCS consistently achieves superior utility (see Section VI).

### G. Extend to Multi-Dimensional Scenarios

The workflow of the multi-dim PrivCS is as follows.

**Step I: Estimating $m$ 1-dim overall CS trees.** Each party allocates half of its users to estimate 1-dim distributions. Concretely, party $P_i$ assigns $|U^i|/2m$ users to each attribute, collects and uploads the noisy histograms to the server, which aggregates these histograms and constructs an overall CS tree for each attribute via the same procedure as in 1-dim settings.

**Step II: Estimating $\binom{m}{2}$ 2-dim adaptive grids.** For each attribute pair $(A_\rho, A_\nu)$ $(\rho \neq \nu)$, party $P_i$ uses $|U^i|/2\binom{m}{2}$ of the remaining users to estimate cell frequencies via OUE with $\epsilon$. The grid partitions are then adaptively determined based on the 1-dim overall CS-tree partitions from Step I. Each party uploads its estimated grids to the server for aggregation.

**Step III: Collaborative frequency filtering aggregation.** After receiving grids from all parties, the server applies CF for aggregation. Unlike the 1-dim case, we construct each 2-dim grid for an attribute pair $(A_\rho, A_\nu)$ based on their 1-dim partitions, since SW is designed for 1-dim estimation. Accordingly, we adapt Equations (6)–(8) to the 2-dim setting by approximating the joint distribution with the corresponding 1-way marginals. Specifically, when computing CF, we set $\beta = |\hat{f}^i_\rho \cdot \hat{f}^i_\nu - \hat{f}_\rho \cdot \hat{f}_\nu|/\hat{f}_\rho \cdot \hat{f}_\nu$ and plug it into Equation (6), where $\hat{f}^i_\rho$ (resp., $\hat{f}_\rho$) is the local (resp., overall) marginal frequency of the current interval for $A_\rho$ estimated in Step I (and similarly for $A_\nu$). We apply the same approximation in Equations (7) and (8) for threshold selection. Finally, we optimize the frequencies via the post-processing method [3].

**Step IV: Answering $\lambda$-dim federated range queries.** We answer a 2-dim query by a $|\mathcal{X}_\rho| \times |\mathcal{X}_\nu|$ response matrix, which encodes the overall 2-dim distribution and is estimated from 1-dim trees and 2-dim grids. Following the previous studies [2], [3], [7], we also apply the maximum entropy and weighted update algorithms, and estimate the distribution using a $2^\lambda$ response matrix derived from the associated 2-dim queries for $\lambda$-dim ($\lambda > 2$) queries.

We then prove that the PrivCS mechanism for multi-dimensional settings satisfies $\epsilon$-LDP, as shown in Theorem 4.

**Theorem 4.** *The PrivCS mechanism for multi-dimensional settings satisfies $\epsilon$-LDP.*

*Proof sketch.* The privacy analysis for users estimating 1-D tree node frequencies follows Theorem 2. For 2-D grid estimation, each user is randomly assigned to one grid and reports a perturbed value via OUE with privacy budget $\epsilon$. All further operations on party- and server-side are based on the noisy data that enjoy the post-processing property. Therefore, PrivCS in multi-dimensional settings satisfies $\epsilon$-LDP. □

## VI. Experiments

### A. Experimental Setting

**Datasets.** We evaluate on six datasets (five attributes each): three real-world datasets (*Amazon*, *Instacart*, *Tokyo*) and three synthetic datasets with different distributions (*Gaussian*, *Zipf*, *Hybrid*); see Table III for a summary. For *Amazon* (Fashion) [44], we use store, timestamp, comment, price, and rating. For *Instacart* [45], we use product, add-to-cart count, department, order hour, and aisle id. For *Tokyo* [46], we use venue, venue category, UTC time, latitude, and longitude. Categorical attributes are mapped to ordinal ones (e.g., converted to comment length to answer how many users are likely to leave a long comment on a store) for evaluation. For synthetic data, *Gaussian* has 3 parties, *Zipf* has 4 parties, and *Hybrid* has 6 parties with 4 mixture distributions; detailed settings are in Appendix D. Unless the original domain is smaller, we discretize each attribute into 1024 or 256 groups for 1-dim or multi-dim experiments, align with existing studies [2], [3], [5], [7], [8].

TABLE III: Datasets.

| Dataset | Description | # Users in Each Party | # Total Users |
|---|---|---|---|
| Amazon | comments and ratings | 415,271 | 1,536,684 |
| | | 877,368 | |
| | | 244,045 | |
| Instacart | grocery orders | 895,832 | 3,346,083 |
| | | 844,162 | |
| | | 803,873 | |
| | | 802,216 | |
| Tokyo | check-ins | 300,203 | 382,409 |
| | | 82,206 | |
| Gaussian | Gaussian $(0, \sigma^2)$, $\sigma^2 \in [1, 10]$ | 50,000 | 300,000 |
| | | 100,000 | |
| | | 150,000 | |
| Zipf | Zipf $(\alpha)$, $\alpha \in [1.01, 1.5]$ | 9,746 | 80,677 |
| | | 17,900 | |
| | | 23,824 | |
| | | 29,207 | |
| Hybrid | Gaussian $(0, \sigma^2)$, $\sigma^2 \in [1, 1.5]$, Zipf (1.01), Gamma (1.5, 1000), and Cauchy (1.0) | 30,000 | 909,000 |
| | | 29,000 | |
| | | 100,000 | |
| | | 100,000 | |
| | | 150,000 | |
| | | 500,000 | |

**Baselines and parameter setting.** We adapt the state-of-the-art range query algorithms under LDP to the federated setting as baselines, including AHEAD [5], PrivNUD [2], PriPL-Tree [3] for 1-dim cases, and HDG [7], AHEAD, PrivNUD, PRISM [8], PriPL-Tree for multi-dim cases. Specifically, we run the baselines in each party and answer the queries by summing up the estimated counts of all parties according to their user amounts and then dividing by the overall population. Following the existing studies [2], [3], we exclude HH, HIO, and DHT, as they have been demonstrated to be inferior to others in 1-dim and multi-dim cases [3], [5], [6], [7]. We use the default hyperparameter settings from the original papers for all baselines. For example, we set the user allocation ratio $\alpha = 0.2$ and the acceleration granularity factor $\phi = 127$ for PriPL-Tree, and $\alpha_1 = 0.7$ and $\alpha_2 = 0.03$ for HDG to compute the optimal 1-D and 2-D grid granularities. In our method, we set $\alpha = 0.3$ being the user proportion for Phase I.

**Metric.** We evaluate the utility of the mechanisms using the mean square error (MSE) metric following the previous studies [2], [3], [5]. Let $Q \in \mathcal{Q}$ denote a federated range query $Q$ in the query set $\mathcal{Q}$, $\hat{f}_Q$ and $f_Q$ are the estimated frequency and ground truth frequency of $Q$, the MSE of $Q$ is calculated as follows, $MSE(\mathcal{Q}) = \sum_{Q \in \mathcal{Q}} \frac{(\hat{f}_Q - f_Q)^2}{|\mathcal{Q}|}$. We randomly generate $1,000$ queries, each of which $Q$ has a random query volume $vol(Q)$, which represents the ratio of

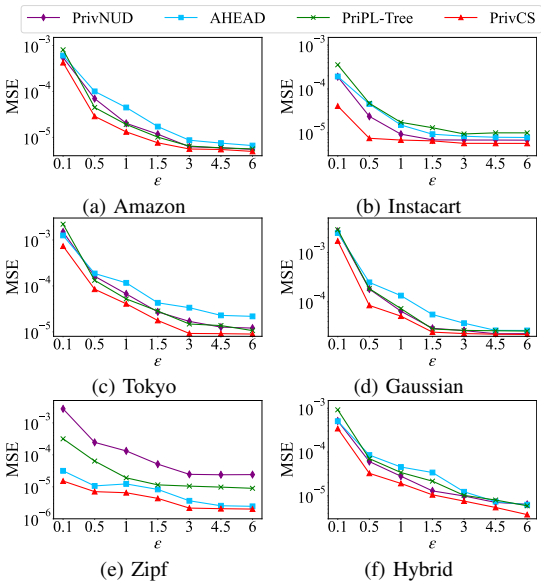



Fig. 4: Impact of $\epsilon$ on 1-dim federated range queries.

the query range size to the domain size on each attribute. We calculate the MSE of these queries, where smaller scores indicate better performance. All the mechanisms are executed 50 times and the average is plotted.

### *B. Evaluation Results on 1-dim Query*

In this subsection, we evaluate the performance of the proposed PrivCS mechanism for 1-dim federated range queries.

**Overall results on utility.** We evaluate PrivCS against baselines on six datasets with $\epsilon$ ranging from 0.1 to 6. Figure 4 shows that AHEAD performs worst on most datasets due to its fixed decomposition granularity. PrivNUD and PriPL-Tree use adaptive decomposition, but their uniform/linear assumptions lead to large errors under skewed, heterogeneous federated distributions. In contrast, PrivCS consistently achieves the best utility, with larger improvements at smaller $\epsilon$. For example, at $\epsilon = 0.5$, PrivCS improves over the best baseline by 30.6% on *Instacart*, 46% on *Hybrid*, and over 26% on *Tokyo*. Similar improvements persist across other datasets and privacy budgets (e.g., 20.7% on *Gaussian*, 24.7% on *Amazon*, and 17.9% on *Zipf* at $\epsilon \in \{1, 1.5, 3, 4.5, 6\}$), showing its effectiveness under different heterogeneous settings.

**Impact study on user allocation ratio.** We investigate the performance of the PrivCS mechanism under different user allocation ratios $\alpha$ for Phase I. As shown in Figure 5 (a), we vary $\alpha$ from 0.1 to 0.9 and run PrivCS on *Zipf*, *Gaussian*, and *Hybrid* datasets. It can be observed that the performance is stable under different $\alpha$, and the optimal performance occurs when $\alpha = 0.3$ (which is the default) or 0.4, illustrating the robustness of the PrivCS against $\alpha$.

**Impact study on domain size.** We evaluate PrivCS with baselines on *Gaussian* dataset of different domain sizes. Figure 5 (b) shows that PrivCS outperforms baselines under varying domain sizes. The private CS fitting and CF aggregation

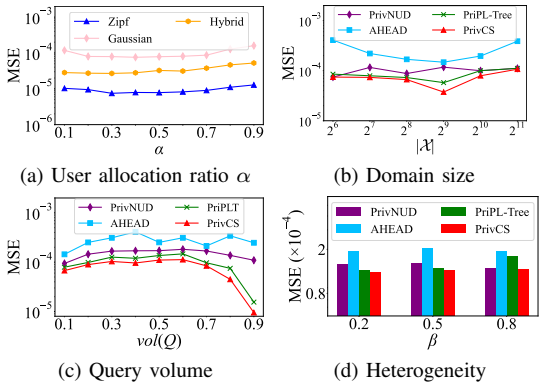



Fig. 5: Impact of parameters on 1-dim queries ($\epsilon = 0.7$).

benefit the performance by providing a better approximation of proportion information of different potent queries within a node and being carefully aggregated across different parties.

**Impact study on query volume.** As shown in Figure 5 (c), we assess the utility of our PrivCS mechanism under varying query volumes $vol(Q)$ from 0.1 to 0.9 on *Gaussian*. The private CS fitting and the CF aggregation enable PrivCS to outperform baselines. The MSE increases when $vol(Q)$ becomes larger and closer to 0.5, as both the ground truth and the estimated frequencies are small, then with increasing $vol(Q)$, the MSE falls quickly since the large volume requires more noisy nodes involved.

**Impact study on heterogeneity.** We study the impact of the data heterogeneity. We use the Dirichlet distribution with different parameters $\beta \in \{0.2, 0.5, 0.8\}$ to assign the overall distribution to each party with different proportions to simulate varying data skewness. A small $\beta$ indicates more unbalanced distributions among parties. In Figure 5 (d), we observe that PrivCS consistently outperforms baselines under different distribution skewnesses. The superiority is mainly because the private CS fitting captures the distribution information efficiently and the CF aggregation leads to less noisy estimated frequencies after aggregation, which takes the distribution imbalance across parties into account.

**Impact study on different party numbers.** We study the impact of different party numbers. We conduct experiments on *Gaussian* dataset with party number $|\mathcal{P}| \in \{2, 4, 6, 8\}$. As shown in Figure 7 (b), PrivCS outperforms all baselines under varying number of parties. Specifically, the improvement over the best baseline is 18.2%, 22.3%, 10.8%, and 11.6% when $|\mathcal{P}| \in \{2, 4, 6, 8\}$, respectively, demonstrating the robustness and effectiveness of the private CS fitting and CF aggregation.

**Impact study on the CS tree.** To investigate the effectiveness of the CS tree, we adapt PrivCS to the traditional single-party LDP setting, namely, without CF aggregation. We conduct experiments on a party of *Gaussian* dataset. As shown in Figure 7 (a), our CS tree outperforms all baselines, since the approximation error is much smaller. We also treat OUE [11] as a baseline in this single-party setting. That is, we estimate the entire histogram and answer a query by summing the

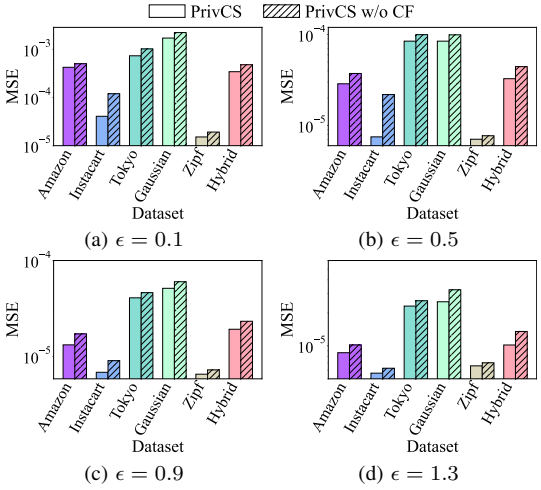



Fig. 6: Impact of CF on 1-dim federated range queries.

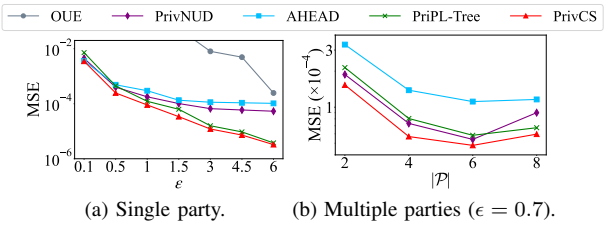



Fig. 7: Impact of party numbers on 1-dim queries.

frequencies of all queried bins. We observe that in Figure 7 (a), **such a flat method cannot achieve reasonable performance**, since the error is proportional to the queried range.

TABLE IV: Server-side communication costs.

| | PrivNUD | AHEAD | PriPL-Tree | OUE | PrivCS |
|---|---|---|---|---|---|
| Comm. | 19.60 KB | 23.92 KB | 78.65 KB | 2,457,600 KB | 58.65 KB |

We evaluate server-side communication on the *Gaussian* dataset. Table IV shows that naive OUE is much more costly than other methods due to the large user population and domain size. Our three-tier design reduces server-side overhead relative to existing range-query baselines. PrivCS incurs more communication than PrivNUD and AHEAD because it requires Phase I histogram estimation (as does PriPL-tree), but achieves higher utility; moreover, private CS fitting reduces communication compared with PriPL-tree. Compression is effective for sparse vectors, yet OUE perturbation often produces dense vectors (depending on the flip probability), making compression non-trivial; we leave this to future work.

**Impact study on the collaborative frequency filtering aggregation.** We investigate the effectiveness of the CF aggregation over different datasets, and the results are shown in Figure 6. PrivCS consistently achieves lower MSE compared to scenarios without the CF aggregation (i.e., excluding the calibration and pruning) across all the datasets. It can be observed that the gap is relatively large under *Hybrid*, *Amazon*, and *Instacart*, which involve more users in the experiments.

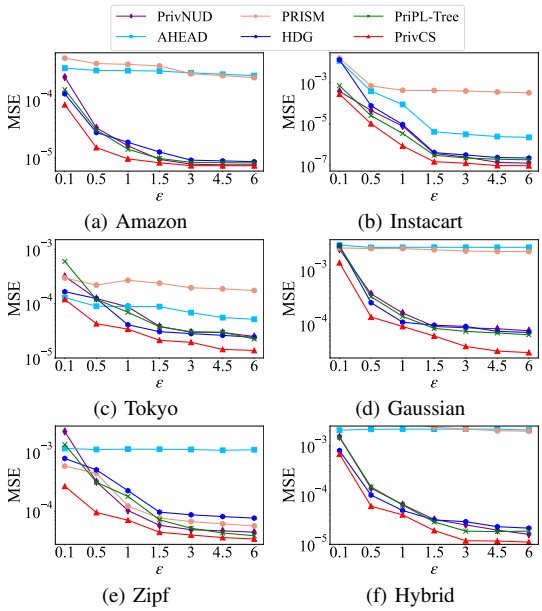



Fig. 8: Impact of $\epsilon$ on multi-dim federated range queries.

This is because the filtering threshold and the frequencies to be pruned are estimated more accurately, given that the variance of the frequency estimation protocol is conversely proportional to the involved user population.

### C. Evaluation Results on Multi-dim Query

In this subsection, we evaluate the performance of our PrivCS mechanism for multi-dim queries. Due to the page limit, we provide the impact study on multi-dim queries in Appendix E.

**Overall results on utility.** We compare PrivCS with HDG, AHEAD, PrivNUD, PRISM, and PriPL-Tree on real-world and synthetic 5-dim datasets. We randomly generate $1,000$ 2-dim queries. As shown in Figure 8, PrivCS outperforms baselines under different datasets. AHEAD and PRISM suffer from the non-adaptive decomposition, which introduces even more errors in federated settings. Compared to HDG, PrivNUD, and PriPL-Tree, PrivCS benefits from the private CS fitting and the CF aggregation that provide more reasonable approximation and reduce the negative impact of noisy frequencies as much as possible, thus achieving better performance.

## VII. Conclusion

In this paper, we study the problem of answering federated range queries over arbitrary data distributions under LDP. We propose a novel private cubic spline (PrivCS) mechanism, in which a private CS fitting is designed to model the underlying data distribution using cubic splines instead of relying on a uniform or linear assumption. We design a novel collaborative frequency filtering aggregation to calibrate and mask certain frequencies, reducing the accumulated errors. To the best of our knowledge, this is the first to explore federated range queries in cross-party settings under the stringent $\epsilon$-LDP.

## Acknowledgment

This work was supported by the National Natural Science Foundation of China (Grant No: 62372122, and 62502416), the Research Grants Council (Grant No: 15208923, 25207224, and 15208825), Hong Kong SAR, China.

## AI-Generated Content Acknowledgement

No AI tools were used to generate the text, figures, images, or pseudo-code in this paper.

## References

[1] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What can we learn privately?" in *SIAM Journal on Computing (SICOMP)*, vol. 40, no. 3, 2011, pp. 793–826.

[2] N. Wang, Y. Wang, Z. Wang, J. Nie, Z. Wei, P. Tang, Y. Gu, and G. Yu, "Privnud: Effective range query processing under local differential privacy," in *ICDE*, 2023, pp. 2660–2672.

[3] L. Wang, Q. Ye, H. Hu, and X. Meng, "Pripl-tree: Accurate range query for arbitrary distribution under local differential privacy," in *PVLDB*, 2024, p. 3031–3044.

[4] T. Wang, B. Ding, J. Zhou, C. Hong, Z. Huang, N. Li, and S. Jha, "Answering multi-dimensional analytical queries under local differential privacy," in *SIGMOD*, 2019, p. 159–176.

[5] L. Du, Z. Zhang, S. Bai, C. Liu, S. Ji, P. Cheng, and J. Chen, "Ahead: Adaptive hierarchical decomposition for range query under local differential privacy," in *CCS*, 2021, p. 1266–1288.

[6] G. Cormode, T. Kulkarni, and D. Srivastava, "Answering range queries under local differential privacy," in *PVLDB*, 2019, p. 1126–1138.

[7] J. Yang, T. Wang, N. Li, X. Cheng, and S. Su, "Answering multi-dimensional range queries under local differential privacy," in *PVLDB*, 2020, p. 378–390.

[8] Y. Wang and X. Cheng, "Prism: Prefix-sum based range queries processing method under local differential privacy," in *ICDE*, 2022, pp. 433–445.

[9] J. Shao, S. Han, C. He, and B. Luo, "Privacy-preserving federated heavy hitter analytics for non-iid data," in *ICML Workshop on Federated Learning and Analytics in Practice*, 2023.

[10] Y. Zhang, Q. Ye, and H. Hu, "Federated heavy hitter analytics with local differential privacy," in *SIGMOD*, vol. 3, no. 1, 2025, pp. 1–27.

[11] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally differentially private protocols for frequency estimation," in *USENIX Security*, 2017, pp. 729–745.

[12] S. L. Warner, "Randomized response: A survey technique for eliminating evasive answer bias," in *Journal of the American Statistical Association (JASA)*, vol. 60, no. 309, 1965, pp. 63–69.

[13] P. Kairouz, S. Oh, and P. Viswanath, "Extremal mechanisms for local differential privacy," in *NeurIPS*, 2014, p. 2879–2887.

[14] T. Wang, N. Li, and S. Jha, "Locally differentially private heavy hitter identification," in *TDSC*, vol. 18, no. 2, 2019, pp. 982–993.

[15] G. Cormode, S. Maddock, and C. Maple, "Frequency estimation under local differential privacy," in *PVLDB*, 2021, p. 2046–2058.

[16] X. Li, W. Liu, J. Lou, Y. Hong, L. Zhang, Z. Qin, and K. Ren, "Local differentially private heavy hitter detection in data streams with bounded memory," in *SIGMOD*, vol. 2, no. 1, 2024, pp. 1–27.

[17] R. McClarren, *Computational Nuclear Engineering and Radiological Science using Python*. Academic Press, 2017.

[18] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *TCC*, 2006, p. 265–284.

[19] N. Li and Q. Ye, "Mobile data collection and analysis with local differential privacy," in *MDM*, 2019, pp. 4–7.

[20] W. Zhu, P. Kairouz, B. McMahan, H. Sun, and W. Li, "Federated heavy hitters discovery with differential privacy," in *AISTATS*, 2020, pp. 3837–3847.

[21] W. Dong, J. Fang, K. Yi, Y. Tao, and A. Machanavajjhala, "R2t: Instance-optimal truncation for differentially private query evaluation with foreign keys," in *SIGMOD*, 2022, pp. 759–772.

[22] W. Dong and K. Yi, "Residual sensitivity for differentially private multi-way joins," in *SIGMOD*, 2021, pp. 432–444.

[23] Y. Hu, H. Ding, and W. Dong, "N2e: A general framework to reduce node-differential privacy to edge-differential privacy for graph analytics," in *SIGMOD*, vol. 3, no. 6, 2025, pp. 1–26.

[24] Q. Ye, L. Yu, K. Huang, X. Xiao, W. Liu, and H. Hu, "From randomized response to randomized index: Answering subset counting queries with local differential privacy," in *S&P*, 2025, pp. 3877–3891.

[25] Q. Ye, H. Hu, X. Meng, and H. Zheng, "Privkv: Key-value data collection with local differential privacy," in *S&P*, 2019, pp. 317–331.

[26] Y. Du, Y. Hu, Z. Zhang, Z. Fang, L. Chen, B. Zheng, and Y. Gao, "Ldptrace: Locally differentially private trajectory synthesis," in *PVLDB*, 2023, p. 1897–1909.

[27] Y. Zhang, Q. Ye, R. Chen, H. Hu, and Q. Han, "Trajectory data collection with local differential privacy," in *PVLDB*, 2023, pp. 2591–2604.

[28] X. Chen, D. Sun, Q. Xu, and W. Dong, "A general framework for per-record differential privacy," in *SIGMOD*, vol. 3, 2025, pp. 1–27.

[29] D. Sun, W. Dong, Y. Qiu, K. Yi, and G. Cormode, "Sum estimation under personalized local differential privacy," in *NeurIPS*, 2025.

[30] M. Hay, V. Rastogi, G. Miklau, and D. Suciu, "Boosting the accuracy of differentially private histograms through consistency," in *PVLDB*, 2010, p. 1021–1032.

[31] W. Qardaji, W. Yang, and N. Li, "Differentially private grids for geospatial data," in *ICDE*, 2013, pp. 757–768.

[32] X. Xiao, G. Wang, and J. Gehrke, "Differential privacy via wavelet transforms," in *TKDE*, vol. 23, no. 8, 2010, pp. 1200–1214.

[33] G. Cormode, C. Procopiuc, D. Srivastava, E. Shen, and T. Yu, "Differentially private spatial decompositions," in *ICDE*, 2012, pp. 20–31.

[34] C. Li, M. Hay, G. Miklau, and Y. Wang, "A data- and workload-aware algorithm for range queries under differential privacy," in *PVLDB*, 2014, p. 341–352.

[35] J. Zhang, X. Xiao, and X. Xie, "Privtree: A differentially private algorithm for hierarchical decompositions," in *SIGMOD*, 2016, p. 155–170.

[36] Z. Huang and K. Yi, "Approximate range counting under differential privacy," in *37th International Symposium on Computational Geometry (SoCG)*, 2021, pp. 45:1–45:14.

[37] C. Dwork, A. Roth *et al.*, "The algorithmic foundations of differential privacy," in *TCS*, vol. 9, no. 3–4, 2014, pp. 211–407.

[38] Z. Li, T. Wang, M. Lopuhaä-Zwakenberg, N. Li, and B. Škoric, "Estimating numerical distributions under local differential privacy," in *SIGMOD*, 2020, p. 621–635.

[39] Z. Zhang, T. Wang, N. Li, S. He, and J. Chen, "Calm: Consistent adaptive local marginal for marginal release under local differential privacy," in *CCS*, 2018, p. 212–229.

[40] A. M. Lekkas and T. I. Fossen, "Integral los path following for curved paths based on a monotone cubic hermite spline parametrization," in *IEEE Transactions on Control Systems Technology*, vol. 22, 2014, pp. 2287–2301.

[41] E. E. Catmull and R. Rom, "A class of local interpolating splines," in *Computer Aided Geometric Design*, 1974, pp. 317–326.

[42] W. Dahmen, B. Han, R.-Q. Jia, and A. Kunoth, "Biorthogonal multiwavelets on the interval: Cubic hermite splines," in *Constructive Approximation*, vol. 16, 2000, pp. 221–259.

[43] G. D. Knott, *Interpolating Cubic Splines*. Springer Science & Business Media, 1999, vol. 18.

[44] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, "Bridging language and items for retrieval and recommendation," in *arXiv preprint arXiv:2403.03952*, 2024.

[45] J. Stanley, M. Risdal, Sharathrao, and W. Cukierski, "Instacart market basket analysis," https://kaggle.com/competitions/instacart-market-basket-analysis, 2017, kaggle.

[46] D. Yang, D. Zhang, V. W. Zheng, and Z. Yu, "Modeling user activity preference by leveraging user spatial temporal characteristics in lbsns," vol. 45, no. 1, 2014, pp. 129–142.

[47] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Brooks/Cole, 2010.

[48] C. De Boor, *A Practical Guide to Splines*. springer New York, 1978, vol. 27.

[49] Q. Li, Y. Diao, Q. Chen, and B. He, "Federated learning on non-iid data silos: An experimental study," in *ICDE*, 2022, pp. 965–978.

[50] M. Yurochkin, M. Agarwal, S. Ghosh, K. Greenewald, N. Hoang, and Y. Khazaeni, "Bayesian nonparametric federated learning of neural networks," in *ICML*, 2019, pp. 7252–7261.

## APPENDIX

### A. Post-Processing Steps

In the weighted averaging step, we traverse nodes in the overall CS tree from the leaves to the root, minimizing the frequency variance for each node. For a leaf node $n_k$, which has frequencies $\hat{f}_k$ (from node frequency estimation) and $\tilde{f}_k$ (from the noisy histogram estimation), we update its frequency to $\hat{f}_k = \theta\hat{f}_k + (1-\theta)\tilde{f}_k$, where $\theta = \frac{\mathrm{Var}(\tilde{f}_k)}{\mathrm{Var}(\hat{f}_k)+\mathrm{Var}(\tilde{f}_k)}$, to result in the minimal variance of $Var(\hat{f}_k) = \frac{Var(\hat{f}_k)Var(\tilde{f}_k)}{Var(\hat{f}_k)+Var(\tilde{f}_k)}$. For each non-leaf node $n_k$, we similarly refine its frequency using its estimated frequency $\hat{f}_k$ and the sum of its child nodes' frequencies $\sum_{n_{cd}\in\mathrm{child}(n_k)} \hat{f}_{cd}$. We then apply the frequency consistency step that updates frequencies from the root to leaves. Note that the frequency of the root does not need to be estimated and optimized, since it represents the entire domain so that the frequency is always 1. Given a parent node $n_p$ with optimized frequency $\hat{f}_p \geq 0$ and its child nodes' frequencies $\{\hat{f}'_{cd} | n_{cd} \in \mathrm{child}(p)\}$ that to be optimized, we solve the following equation: $\min \sum_{n_{cd}\in\mathrm{child}(n_p)} \left(\hat{f}_{cd} - \hat{f}'_{cd}\right)^2$, subject to $\sum_{n_{cd}\in\mathrm{child}(n_p)} \hat{f}_{cd} = \hat{f}_p$ and $\forall n_{cd} \in \mathrm{child}(n_p), \hat{f}_{cd} \geq 0$. Let $D^+$ denote the set of child nodes with positive updated frequencies, and $D_0$ be those with zero updated frequencies, then the optimal frequency is $\hat{f}_{cd} = \hat{f}_{cd} + \left(\hat{f}_p - \sum_{n_j\in D^+} \hat{f}_j\right)/|D^+|$ if $n_{cd} \in D^+$, and $\hat{f}_{cd} = 0$ if $n_{cd} \in D_0$.

### B. Privacy Proof

We prove that the proposed PrivCS mechanism satisfies $\epsilon$-LDP.

**Theorem 5.** *The PrivCS mechanism satisfies $\epsilon$-LDP.*

*Proof.* The PrivCS mechanism consists of three phases, private CS fitting, PrivCS tree construction, and collaborative frequency filtering aggregation.

In the first phase, each party requires a subset of its users to estimate the noisy histogram. These users perturb their data locally via SW with the privacy budget $\epsilon$ and upload the perturbed data to their respective parties. The parties then send the estimated histograms to the server. Since SW satisfies $\epsilon$-LDP, the estimated results on both the party and server sides satisfy LDP. Based on the post-processing theorem, fixing the breakpoints and partitions by minimizing the interpolating errors also satisfies $\epsilon$-LDP, as all the parameters used are either derived from noisy results or chosen independently.

In the construction phase, let $N-1$ denote the number of non-root nodes in the CS tree. Each party randomly divides its remaining users into $N-1$ groups, which do not raise privacy concerns. Each user in a different group estimates the frequencies of the corresponding node (representing a different interval) by perturbing their value via OUE with privacy budget $\epsilon$. They then upload the perturbed values to their respective parties, which send the aggregated estimations to the server. This phase satisfies LDP, as OUE itself satisfies LDP and all parameters used in the reduction and user allocation are privacy-free.

In the aggregation phase, we apply the collaborative frequency filtering, weighted averaging, and the frequency consistency step based on independently determined parameters and the noisy results estimated by SW and OUE. Therefore, the aggregation phase also satisfies LDP according to the post-processing theorem. Thus, the proof is complete. □

### C. Utility Proof

Here we prove that CS tree leads to smaller errors than PriPL-Tree under the same target error.

**Theorem 6.** *Given a target uniform error $1 > \delta_u > 0$, let $\mathrm{Var}_{cs}$ (resp. $\mathrm{Var}_{lin}$) denote the variance of frequencies in the CS tree (resp. PriPL-Tree), we have: $\frac{\mathrm{Var}_{\mathrm{cs}}}{\mathrm{Var}_{\mathrm{lin}}} = \Theta\big(\delta_u^{1/4}\big)$.*

*Proof.* By Lemma 1, to achieve a uniform error $\|G_t - L\|_\infty \leq \delta_u$ and $\|G_t - H\|_\infty \leq \delta_u$ we must have

$$N_{\mathrm{lin}} \;=\; O\big(\delta_u^{-1/2}\big), \qquad N_{\mathrm{CS}} \;=\; O\big(\delta_u^{-1/4}\big).$$

Then by Lemma 2, the frequency-error variances in the two trees satisfy

$$\mathrm{Var}_{\mathrm{lin}} = O\Big(\frac{N_{\mathrm{lin}}\,\log N_{\mathrm{lin}}}{(1-\alpha)\,|U|\,\epsilon^2}\Big), \quad \mathrm{Var}_{\mathrm{cs}} = O\Big(\frac{N_{\mathrm{CS}}\,\log N_{\mathrm{CS}}}{(1-\alpha)\,|U|\,\epsilon^2}\Big).$$

It follows that

$$\frac{\mathrm{Var}_{\mathrm{cs}}}{\mathrm{Var}_{\mathrm{lin}}} = \Theta\Big(\frac{\delta_u^{-1/4}\,\log(\delta_u^{-1/4})}{\delta_u^{-1/2}\,\log(\delta_u^{-1/2})}\Big) = \Theta\big(\delta_u^{1/4}\big),$$

which establishes the theorem. □

**Lemma 1.** *Let $G_t \in C^4([a,b])$ and fix a target uniform error $\delta_u > 0$. Denote by $L$ (resp. $H$) the piecewise–linear (resp. cubic Hermite spline) interpolant of $G_t$ on a uniform mesh of maximum step $h_{\mathrm{lin}}$ (resp. $h_{\mathrm{CS}}$). Set $N_{\mathrm{lin}} = \frac{b-a}{h_{\mathrm{lin}}}$ and $N_{\mathrm{CS}} = \frac{b-a}{h_{\mathrm{CS}}}$ for the number of segments in each approximation. If $\|G_t - L\|_\infty \leq \delta_u$ and $\|G_t - H\|_\infty \leq \delta_u$, then necessarily $N_{\mathrm{lin}} = O\big(\delta_u^{-1/2}\big)$ and $N_{\mathrm{CS}} = O\big(\delta_u^{-1/4}\big)$.*

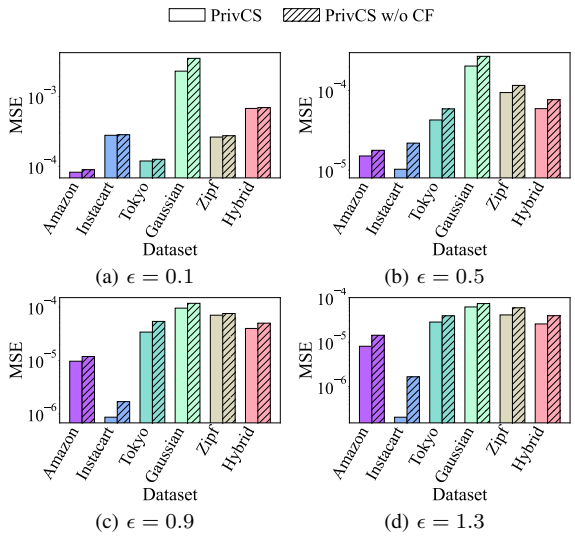
*Proof.* Let $M_2 = \max_{x\in[a,b]} |G_t''(x)|$, $M_4 = \max_{x\in[a,b]} |G_t^{(4)}(x)|$. Here and below, the constants hidden in the $O(\cdot)$ notation depend only on the interval length $b-a$ and on $M_2, M_4$. By the standard error bound for piecewise–linear interpolation [47], we have $\|G_t - L\|_\infty \leq \frac{h_{\mathrm{lin}}^2}{8} M_2 \leq \delta_u$, which yields $h_{\mathrm{lin}} = O\big(\sqrt{\delta_u}\big) \Longrightarrow N_{\mathrm{lin}} = \frac{b-a}{h_{\mathrm{lin}}} = O\big(\delta_u^{-1/2}\big)$. Similarly, the classical estimate for cubic splines [48] gives $\|G_t - H\|_\infty \leq \frac{h_{\mathrm{CS}}^4}{384} M_4 \leq \delta_u$, from which $h_{\mathrm{CS}} = O\big(\delta_u^{1/4}\big) \Longrightarrow N_{\mathrm{CS}} = \frac{b-a}{h_{\mathrm{CS}}} = O\big(\delta_u^{-1/4}\big)$. □

**Lemma 2.** *Given a CS tree or PriPL-Tree with at most $N$ segments (namely leaf nodes), the variance of frequencies is capped at $O(\frac{N\cdot\log N}{(1-\alpha)\cdot|U|\cdot\epsilon^2})$.*

*Proof.* For convenience, assume the CS tree or PriPL-Tree has maximum height $h_{\max} \leq \log_2 N$. Denote by $\alpha_k$ the fraction of users at any node at depth $k$, so that $\alpha_k \geq \frac{1-\alpha}{h_{\max}} \geq \frac{1-\alpha}{\log_2 N}$, and let each non-leaf node at depth $k$ have $b_k$ children with $1 \leq b_k \leq N$.

TABLE V: Synthetic dataset details.

| Dataset | Dimension 1 | Dimension 2 | Dimension 3 | Dimension 4 | Dimension 5 | # Users | # Total Users |
|---|---|---|---|---|---|---|---|
| Gaussian | Gaussian(0, 2) | Gaussian(0, 5) | Gaussian(0, 2) | Gaussian(0, 5) | Gaussian(0, 10) | 50,000 | 300,000 |
| | Gaussian(0, 1) | Gaussian(0, 7) | Gaussian(0, 3) | Gaussian(0, 10) | Gaussian(0, 2) | 150,000 | |
| | Gaussian(0, 1.5) | Gaussian(0, 5) | Gaussian(0, 2) | Gaussian(0, 2) | Gaussian(0, 10) | 150,000 | |
| Zipf | Zipf(1.5) | Zipf(1.3) | Zipf(1.1) | Zipf(1.01) | Zipf(1.5) | 9,746 | 80,677 |
| | Zipf(1.3) | Zipf(1.5) | Zipf(1.3) | Zipf(1.1) | Zipf(1.01) | 17,900 | |
| | Zipf(1.1) | Zipf(1.01) | Zipf(1.5) | Zipf(1.3) | Zipf(1.1) | 23,824 | |
| | Zipf(1.01) | Zipf(1.1) | Zipf(1.01) | Zipf(1.5) | Zipf(1.3) | 29,207 | |
| Hybrid | Cauchy(1.0) | Cauchy(1.0) | Cauchy(1.0) | Cauchy(1.0) | Cauchy(1.0) | 30,000 | 909,000 |
| | Zipf(1.01) | Zipf(1.01) | Zipf(1.01) | Zipf(1.01) | Zipf(1.01) | 29,000 | |
| | Gamma(1.5, 1000) | Gamma(1.5, 1000) | Gamma(1.5, 1000) | Gamma(1.5, 1000) | Gamma(1.5, 1000) | 100,000 | |
| | Gaussian(0, 1.5) | Gaussian(0, 1.5) | Gaussian(0, 1.5) | Gaussian(0, 1.5) | Gaussian(0, 1.5) | 100,000 | |
| | Gaussian(0, 1) | Gaussian(0, 1) | Gaussian(0, 1) | Gaussian(0, 1) | Gaussian(0, 1) | 150,000 | |
| | Gamma(10, 1000) | Gamma(10, 1000) | Gamma(10, 1000) | Gamma(10, 1000) | Gamma(10, 1000) | 500,000 | |




(a) $\epsilon = 0.1$ (b) $\epsilon = 0.5$ (c) $\epsilon = 0.9$ (d) $\epsilon = 1.3$

Fig. 9: Impact of CF on multi-dim federated range queries.

For each leaf node $n_k$, let $\hat{f}_s$ be the SW estimate (using $\alpha|U|$ users) and $\hat{f}_o$ the OUE estimate (using $\alpha_k|U|$ users). The weighted-average estimate $\hat{f}$ has variance $Var(\hat{f}) = \frac{Var(\hat{f}_s)\,Var(\hat{f}_o)}{Var(\hat{f}_s)+Var(\hat{f}_o)} = Var(\hat{f}_o) - \frac{Var^2(\hat{f}_o)}{Var(\hat{f}_s)+Var(\hat{f}_o)} \leq Var(\hat{f}_o)$. Moreover, $Var(\hat{f}_o) = \frac{4e^{\epsilon}}{|U|\,\alpha_k(e^{\epsilon}-1)^2} = O((|U|\alpha_k\epsilon^2)^{-1}) = O\big(\frac{\log N}{(1-\alpha)\,|U|\,\epsilon^2}\big)$. Hence $Var(\hat{f}) = O\big(\frac{\log N}{(1-\alpha)\,|U|\,\epsilon^2}\big)$.

Consider any internal node $n_p$ with $b_p$ children. Let $\hat{f}_o^p$ be its own OUE estimate, and write $S_p = \sum_{n_c \in \text{child}(n_p)} \hat{f}^c$. Then $Var(\hat{f}^p) = \frac{Var(\hat{f}_o^p)\,Var(S_p)}{Var(\hat{f}_o^p)+Var(S_p)} = Var(\hat{f}_o^p) - \frac{Var^2(\hat{f}_o^p)}{Var(\hat{f}_o^p)+Var(S_p)}$. Since $Var(S_p) = \sum_{n_c} Var(\hat{f}^c) \leq b_p\,Var(\hat{f}_o^p)$, it follows that $Var(\hat{f}^p) \leq \frac{b_p}{b_p+1} Var(\hat{f}_o^p) \leq \frac{N}{N+1} Var(\hat{f}_o^p) = O\big(\frac{N\log N}{(1-\alpha)\,|U|\,\epsilon^2}\big)$.

Fix a parent $n_p$ and one child $n_c$. Partition its children into $D_+ = \{n_s : \hat{f}_s > 0\}$ and $D_0 = \{n_s : \hat{f}_s \leq 0\}$. If $n_c \in D_+$ then the corrected estimate is $\hat{f}'_c = \frac{|D_+|-1}{|D_+|}\hat{f}_c + \frac{1}{|D_+|}\hat{f}^p - \frac{1}{|D_+|}\sum_{n_s \in D_+\setminus\{n_c\}} \hat{f}_s$. By expanding variances and Cauchy–Schwarz one shows $Var(\hat{f}'_c) \leq \frac{(|D_+|-1)^2}{|D_+|^2} Var(\hat{f}_c) + \frac{1}{|D_+|^2} Var(\hat{f}^p) + \frac{1}{|D_+|^2}\sum_{n_s \in D_+\setminus\{n_c\}} Var(\hat{f}_s) + \frac{2(|D_+|-1)}{|D_+|^2}\sqrt{Var(\hat{f}^p)Var(\hat{f}_c)} \leq \frac{|D_+|+1}{|D_+|}\max\{Var(\hat{f}^p)\} \cup \{Var(\hat{f}_j)\}$. Iterating this up to depth $h_{\max}$ multiplies the bound by at most $(\frac{|D_+|+1}{|D_+|})^{h_{\max}} \leq 2^{\log_2 N} = N$, so $Var(\hat{f}'_c) = O\big(\frac{N\log N}{(1-\alpha)\,|U|\,\epsilon^2}\big)$.

Combining the above results yields the claimed bound. □

### D. Datasets

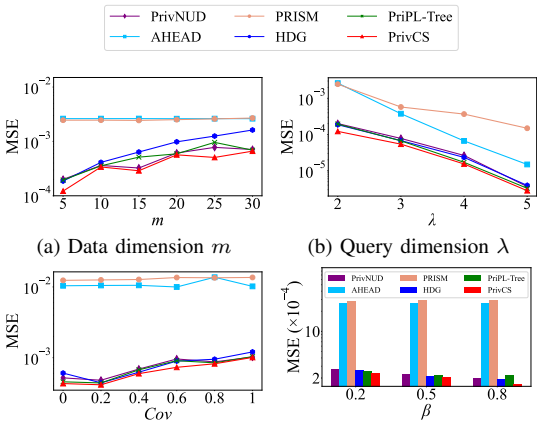
We conduct experiments over three real-world datasets (*Amazon*, *Instacart*, and *Tokyo*) and three synthetic datasets following different data distributions (*Gaussian*, *Zipf*, and *Hybrid*), each comprising 5 attributes.

For the *Amazon* review dataset [44], we select stores, timestamps, comments, prices, and ratings from the Fashion category. The *Instacart* dataset [45] contains anonymized grocery orders from Instacart users. We choose products, numbers of add to cart, departments, order hours, and aisle ids for analysis and distribute the orders across four parties. *Tokyo* Check-in Dataset [46] contains check-in data collected from April 2012 to February 2013. We choose venues, venue categories, UTC times, latitudes, and longitudes as attributes. Following the existing studies [10], [49], [50], we employ Dirichlet distribution on the frequencies ($\beta = 0.5$) to simulate realistic non-IID data distribution across parties. The concentration parameter $\beta$ controls the degree of the distribution skewness. Due to the small $\beta = 0.5$, some parties' users may not have any values in an attribute's subdomain.

The *Gaussian* dataset has three parties, and the *Zipf* dataset contains four parties. We set the frequency distributions of different dimensions and parties with varying parameters. The *Hybrid* dataset includes six parties and four kinds of distributions. Table V summarizes the details. Aligning with the existing studies [2], [3], [5], [7], [8], we partition each attribute's domain into 1024 groups for 1-dim settings or 256 groups for multi-dim settings, unless the original domain size is smaller than these specified thresholds.

### E. Impact Study on Multi-Dim Query

**Impact study on the collaborative frequency filtering aggregation.** We evaluate if the CF aggregation is effective in answering multi-dim queries. The observation in Figure 9 is similar to that in 1-dim settings. The PrivCS always outperforms the scenario without CF aggregation, demonstrating its effectiveness under more complex multi-dim scenarios.




(a) Data dimension $m$ (b) Query dimension $\lambda$

(c) Covariance (d) Heterogeneity

Fig. 10: Impact of parameters on multi-dim queries ($\epsilon = 0.7$).

**Impact study on data dimension.** We study the impact of data dimension. We run the algorithms on *Gaussian* datasets with different data dimensions $m \in \{5, 10, 15, 20, 25, 30\}$ for answering 2-dim queries. The results shown in Figure 10 (a) demonstrate the robustness of PrivCS, which outperforms all baselines under varying data dimensions. The error grows as $m$ increases since there are fewer users to estimate each node's frequency due to the larger domain.

**Impact study on query dimension.** We evaluate PrivCS with $1,000$ randomly generated $\lambda$-dim queries for $\lambda \in \{2, 3, 4, 5\}$. As shown in Figure 10 (b), PrivCS consistently outperforms the baselines. Moreover, all methods exhibit lower MSE as $\lambda$ increases, since higher-dimensional ranges typically contain fewer records (i.e., lower frequencies).

**Impact study on attribute correlation.** To better understand the impact of the attribute correlation, we conduct experiments on *Gaussian* datasets with covariances spanning $[0, 1]$. As shown in Figure 10 (c), PrivCS consistently outperforms baselines across all correlation levels, illustrating the superiority and robustness of the private CS fitting and CF aggregation.

**Impact study on heterogeneity.** We investigate the influence of data heterogeneity in multi-dim settings as well. Consistent with our 1-dim experiments, we use the Dirichlet distribution with parameters $\beta \in \{0.2, 0.5, 0.8\}$ to generate varying data skewnesses. Figure 10 (d) shows that PrivCS is also robust under multi-dim settings. This is because the CS fitting and CF aggregation effectively reduce the approximation errors and accumulated LDP noises.

**Impact study on the collaborative frequency filtering aggregation.** At last, we evaluate if the CF aggregation is effective in answering multi-dim queries. The observation in Figure 9 is similar to that in 1-dim settings. The PrivCS always outperforms the scenario without CF aggregation, demonstrating its effectiveness under more complex multi-dim scenarios.